In [1]:
```
#importing the necessary libraries
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

In [2]:
```
#import the data
df=pd.read_csv('/kaggle/input/bostan-data/Bostonfinal.csv')
```

In [3]:
```
df.head()
```

Out[3]:

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.9
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.9
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.8
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.6
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.9

In [4]:
```
df.shape
```

Out[4]:
```
(4988, 14)
```

In [5]:
```
df.columns
```

Out[5]:
```
Index(['crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis', 'rad', 'ta
x',
       'ptratio', 'b', 'lstat', 'medv'],
      dtype='object')
```

In [6]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4988 entries, 0 to 4987
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   crim     4988 non-null   float64
 1   zn       4988 non-null   float64
 2   indus    4988 non-null   float64
 3   chas     4988 non-null   int64
 4   nox      4988 non-null   float64
 5   rm       4932 non-null   float64
 6   age      4988 non-null   float64
 7   dis      4988 non-null   float64
 8   rad      4988 non-null   int64
 9   tax      4988 non-null   int64
 10  ptratio  4988 non-null   float64
 11  b        4988 non-null   float64
 12  lstat    4988 non-null   float64
 13  medv     4988 non-null   float64
dtypes: float64(11), int64(3)
memory usage: 545.7 KB
```

In [7]:
```
#checking null values
df.isnull().any()
```

Out[7]:
```
crim       False
zn         False
indus      False
chas       False
nox        False
rm          True
age        False
dis        False
rad        False
tax        False
ptratio    False
b          False
lstat      False
medv       False
dtype: bool
```

we can see there are null values in rm column in our dataset

In [8]:
```
df.isnull().sum()
```

Out[8]:
```
crim        0
zn          0
indus       0
chas        0
nox         0
rm         56
age         0
dis         0
rad         0
tax         0
ptratio     0
b           0
lstat       0
medv        0
dtype: int64
```

Replacing the null values with mean value

In [9]:
```
df['rm']=df['rm'].fillna(df['rm'].mean())
```

In [10]:
```
df.isnull().sum()
```

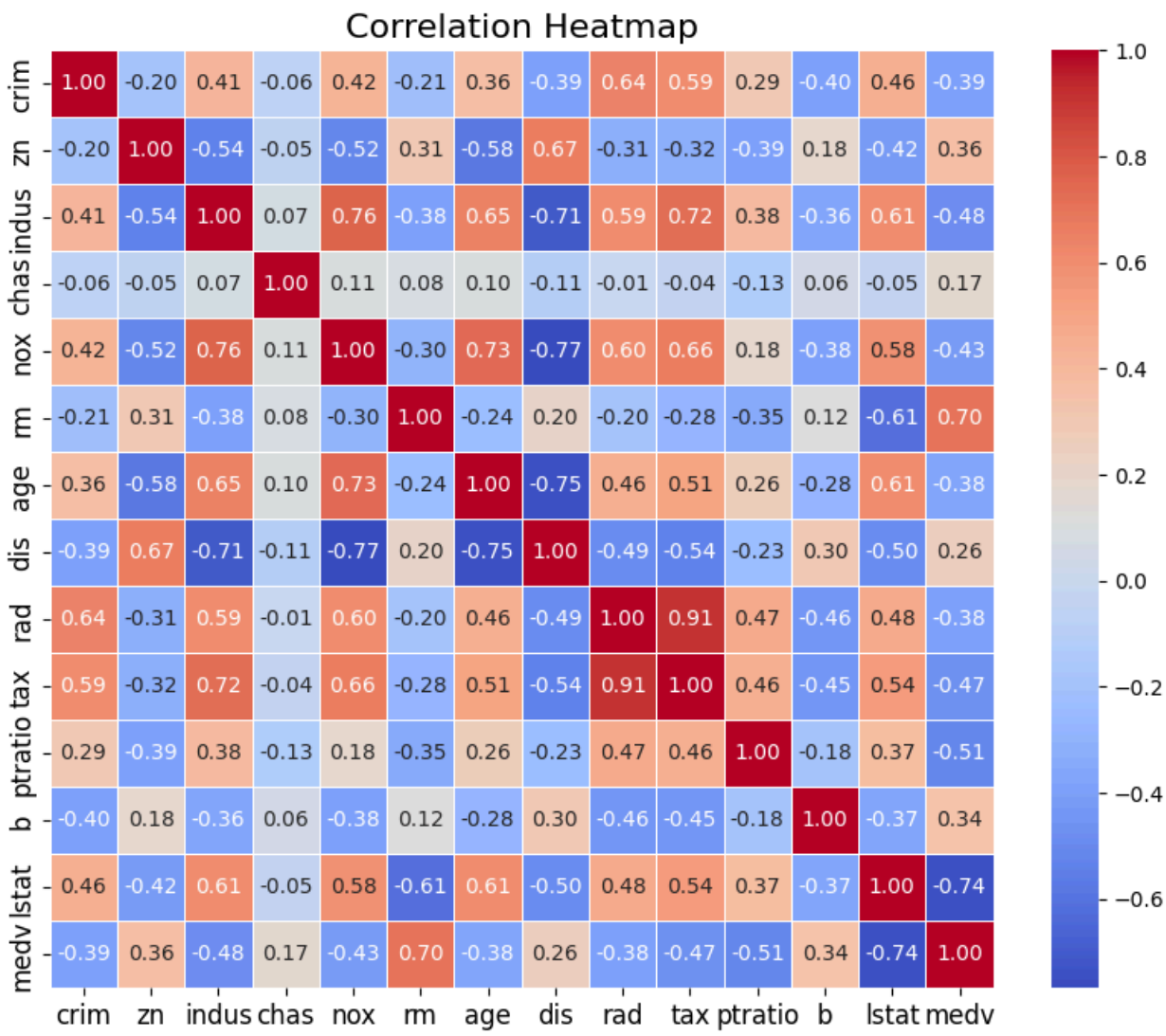
Out[10]:
```
crim       0
zn         0
indus      0
chas       0
nox        0
rm         0
age        0
dis        0
rad        0
tax        0
ptratio    0
b          0
lstat      0
medv       0
dtype: int64
```

we can see that null values are removed

Data Visualization

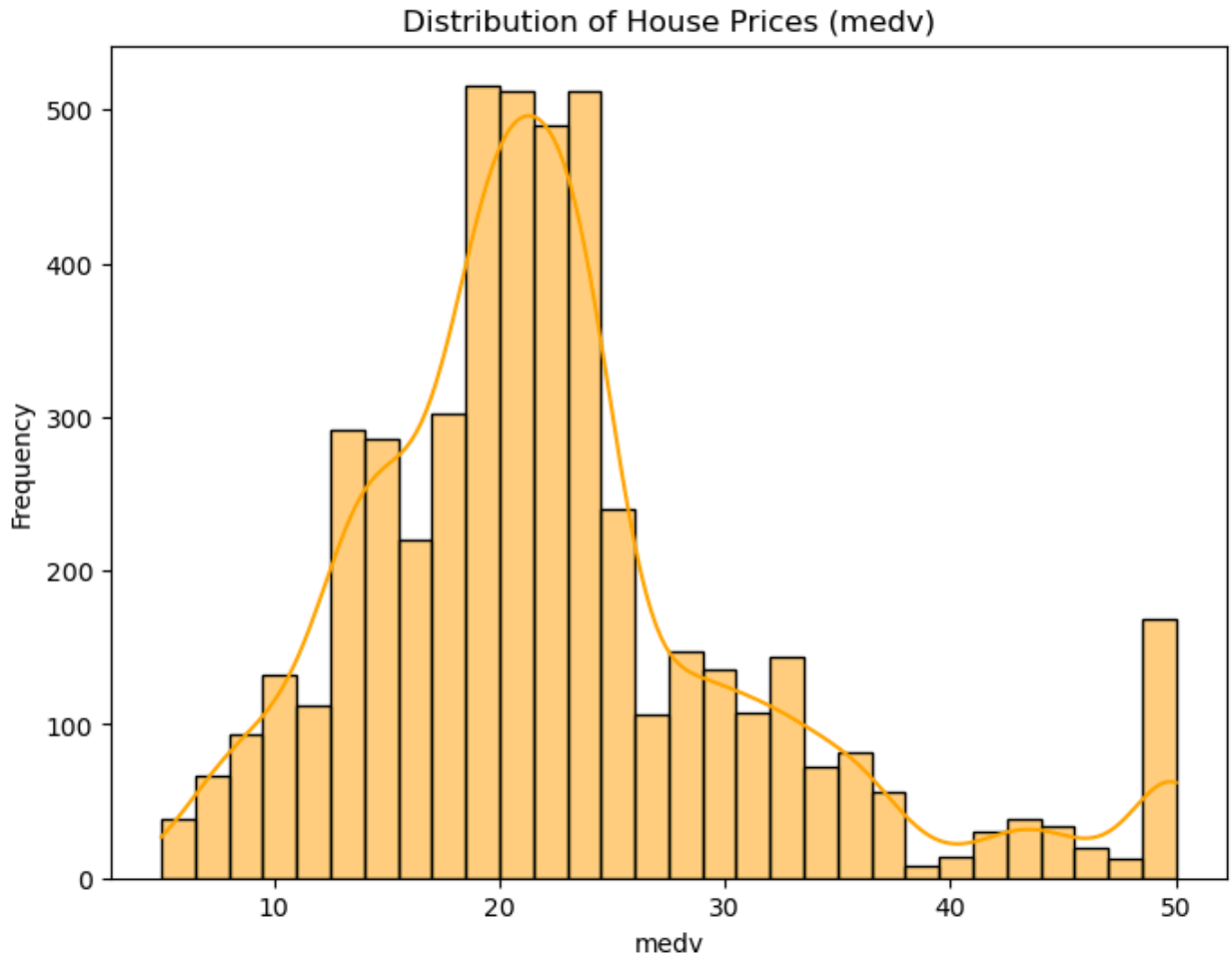
In [11]:
```
corr_matrix = df.corr()
plt.figure(figsize=(10, 8))
```

```
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', linewidths=0.5, fmt='.2f

plt.title('Correlation Heatmap', fontsize=16)

plt.yticks(fontsize=12)
plt.xticks(fontsize=12)

plt.show()
```

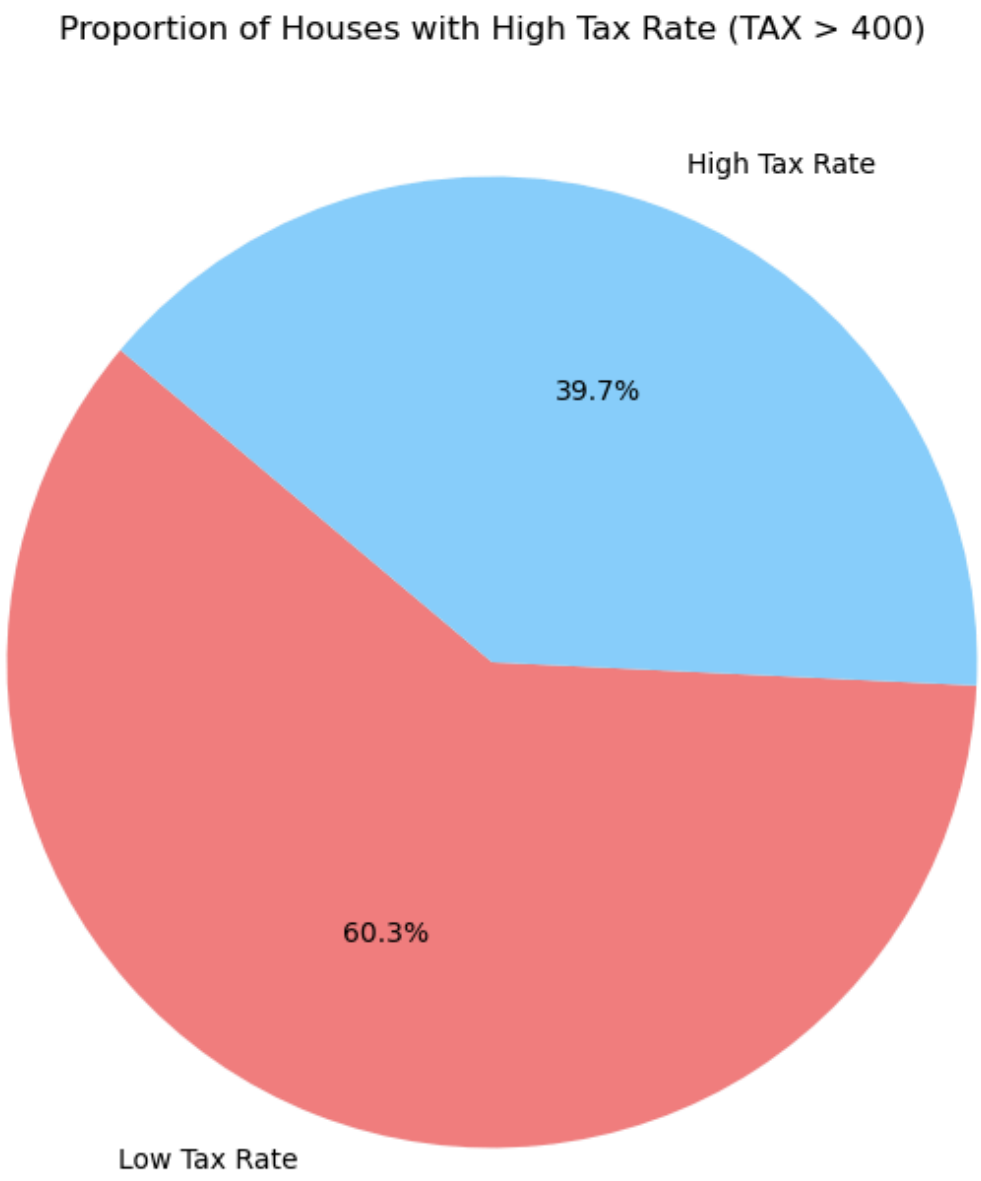


In [20]:

```
# from scipy import stats

# # Visualize distribution of target variable 'medv'
# plt.figure(figsize=(8, 6))
# sns.histplot(df['medv'], bins=30, kde=True, color='orange')
# plt.title('Distribution of House Prices (medv)')
# plt.xlabel('medv')
# plt.ylabel('Frequency')
# plt.show()
```



In [21]:
```
#  #Pie Chart for Proportion of Houses with High Tax Rate (TAX > 400)
# high_tax_count = len(df[df['tax'] > 400])
# low_tax_count = len(df) - high_tax_count
# tax_counts = [low_tax_count, high_tax_count]

# labels = ['Low Tax Rate', 'High Tax Rate']
# colors = ['lightcoral', 'lightskyblue']

# plt.figure(figsize=(8, 8))
# plt.pie(tax_counts, labels=labels, colors=colors, autopct='%1.1f%%', startan
# plt.title('Proportion of Houses with High Tax Rate (TAX > 400)')
# plt.show()
```



In [12]:
```
#
```

In [13]:
```
# # Histogram for the Distribution of Crime Rates (CRIM)
# plt.figure(figsize=(10, 6))
# sns.histplot(df['crim'], bins=30, kde=True, color='red')
# plt.title('Histogram: Distribution of Crime Rates (CRIM)')
# plt.xlabel('Crime Rate (CRIM)')
# plt.ylabel('Frequency')
# plt.show()
```

In [14]:
```
# # Visualizing relationships between features and target variable using scatt
# plt.figure(figsize=(16, 12))
# for i, col in enumerate(df.columns[:-1]):
#     plt.subplot(3, 5, i+1)
```

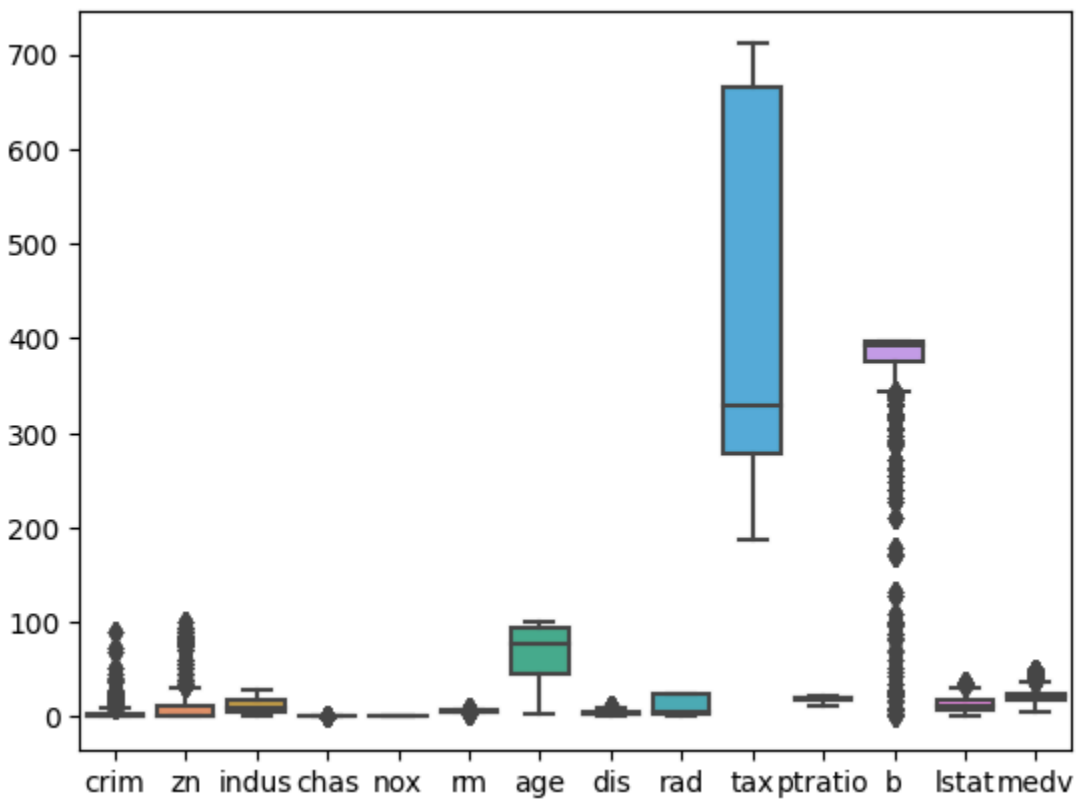
```
#     sns.scatterplot(x=df[col], y=df['medv'], color='green', alpha=0.7)
#     plt.title(f'{col} vs medv')
# plt.tight_layout()
# plt.show()
```

In [15]:
```
# # Line Plot for Median House Price over the Age of Homes (AGE)
# median_price_by_age = df.groupby('age')['medv'].median().reset_index()
# plt.figure(figsize=(12, 6))
# sns.lineplot(x='age', y='medv', data=median_price_by_age, color='purple')
# plt.title('Line Plot: Median House Price over the Age of Homes (AGE)')
# plt.xlabel('Age of Homes (AGE)')
# plt.ylabel('Median Price')
# plt.show()
```

Outlier Detection

In [18]:
```
#box plot
sns.boxplot(data=df)
```

Out[18]: <Axes: >



we can see outliers are present in crim,zn,rm,dis,ptratio,Lstat columns

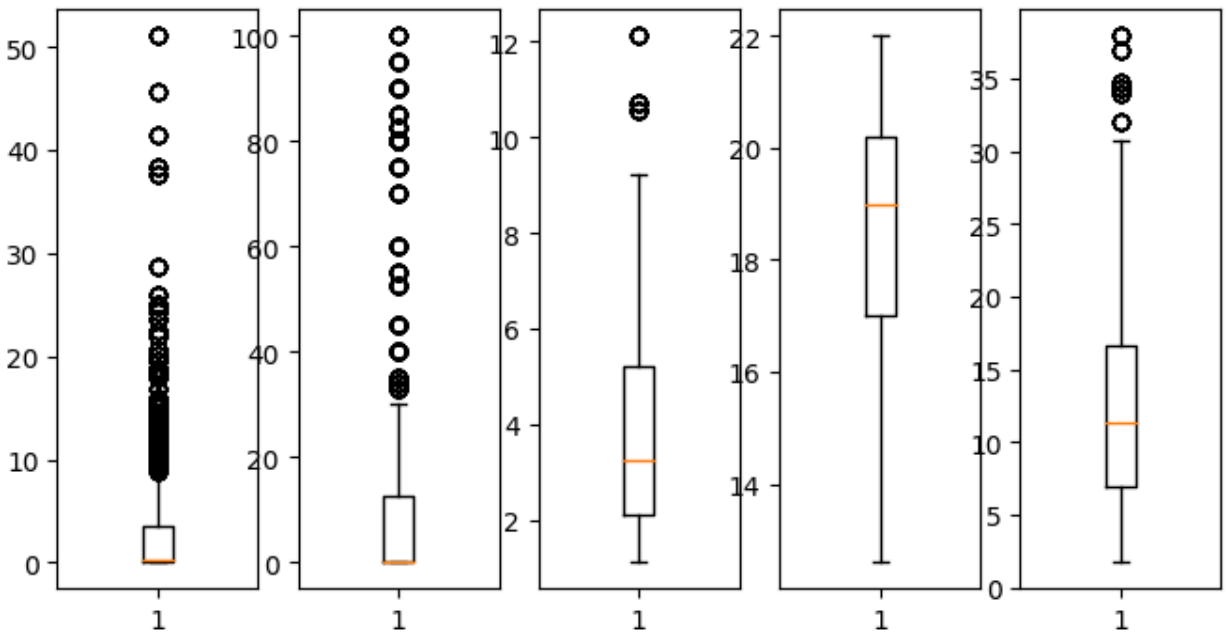
Removing outliers

removing outliers using z-score method

In [19]:
```
from scipy import stats
```

In [20]:
```
# Function to remove outliers using IQR
def remove_outliers_iqr(dataframe, column, lower_bound, upper_bound):
    Q1 = dataframe[column].quantile(lower_bound)
    Q3 = dataframe[column].quantile(upper_bound)
    IQR = Q3 - Q1
    return dataframe[(dataframe[column] >= Q1 - 3 * IQR) & (dataframe[column]

# Specify columns for which you want to remove outliers
columns_to_check = ['crim', 'zn', 'chas', 'rm', 'dis','ptratio', 'b', 'lstat',

# Set the lower and upper bounds for outlier removal (adjust as needed)
lower_bound = 0.05
upper_bound = 0.95

# Remove outliers for each specified column
for column in columns_to_check:
    df = remove_outliers_iqr(df, column, lower_bound, upper_bound)
```

In [21]:
```
plt.figure(figsize=(8,4))
plt.subplot(151)
plt.boxplot(df['crim'])
plt.subplot(152)
plt.boxplot(df['zn'])
plt.subplot(153)
plt.boxplot(df['dis'])
plt.subplot(154)
plt.boxplot(df['ptratio'])
plt.subplot(155)
plt.boxplot(df['lstat'])
plt.show()
```



There are less outliers than before .It means they are removed.

In [22]:
```
df.describe()
```

Out[22]:

	crim	zn	indus	chas	nox	
count	4964.000000	4964.000000	4964.000000	4964.000000	4964.000000	4964.00
mean	3.148931	11.723811	11.094815	0.074134	0.554503	6.28
std	6.381427	23.915497	6.862917	0.262015	0.116258	0.69
min	0.006320	0.000000	0.460000	0.000000	0.385000	3.56
25%	0.079780	0.000000	5.130000	0.000000	0.449000	5.88
50%	0.252775	0.000000	9.690000	0.000000	0.538000	6.21
75%	3.535010	12.500000	18.100000	0.000000	0.624000	6.61
max	51.135800	100.000000	27.740000	1.000000	0.871000	8.78

In [23]:
```
df.shape
```

Out[23]: (4964, 14)

We can see shape and values in description of the data are changed .so outliers are removed

We can see every columns in our data are numeric. so we don't need to apply any encoding techniques

Separating data into dependent and independent variables

In [24]:
```
x=df.drop('medv',axis=1)
```

In [25]:
```
x.columns
```

Out[25]:
```
Index(['crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis', 'rad', 'ta
x',
       'ptratio', 'b', 'lstat'],
      dtype='object')
```

In [26]:
```
y=df['medv']
y
```

Out[26]:
```
0       24.0
1       21.6
2       34.7
3       33.4
4       36.2
        ...
4983    14.5
4984    21.2
4985    23.1
4986    20.0
4987    25.0
Name: medv, Length: 4964, dtype: float64
```

Feature Scaling

In [27]:
```
from sklearn.preprocessing import MinMaxScaler

minmax = MinMaxScaler()
columns = x.columns
x = pd.DataFrame(minmax.fit_transform(x), columns=columns)
```

In [28]:
```
x.head()
```

Out[28]:

	crim	zn	indus	chas	nox	rm	age	dis	ra
0	0.000000	0.18	0.067815	0.0	0.314815	0.577505	0.641607	0.269203	0.0000
1	0.000411	0.00	0.242302	0.0	0.172840	0.547998	0.782698	0.348962	0.0434
2	0.000410	0.00	0.242302	0.0	0.172840	0.694386	0.599382	0.348962	0.0434
3	0.000509	0.00	0.063050	0.0	0.150206	0.658555	0.441813	0.448545	0.0869
4	0.001227	0.00	0.063050	0.0	0.150206	0.687105	0.528321	0.448545	0.0869

In [29]:
```
!pip install sweetviz
```

```
Collecting sweetviz
  Downloading sweetviz-2.3.1-py3-none-any.whl.metadata (24 kB)
Requirement already satisfied: pandas!=1.0.0,!=1.0.1,!=1.0.2,>=0.25.3 in /usr/l
ocal/lib/python3.11/dist-packages (from sweetviz) (2.2.3)
Requirement already satisfied: numpy>=1.16.0 in /usr/local/lib/python3.11/dist-
packages (from sweetviz) (1.26.4)
Requirement already satisfied: matplotlib>=3.1.3 in /usr/local/lib/python3.11/d
ist-packages (from sweetviz) (3.7.2)
Requirement already satisfied: tqdm>=4.43.0 in /usr/local/lib/python3.11/dist-p
ackages (from sweetviz) (4.67.1)
Requirement already satisfied: scipy>=1.3.2 in /usr/local/lib/python3.11/dist-p
ackages (from sweetviz) (1.15.3)
Requirement already satisfied: jinja2>=2.11.1 in /usr/local/lib/python3.11/dis
t-packages (from sweetviz) (3.1.6)
Requirement already satisfied: importlib-resources>=1.2.0 in /usr/local/lib/pyt
hon3.11/dist-packages (from sweetviz) (6.5.2)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/dis
t-packages (from jinja2>=2.11.1->sweetviz) (3.0.3)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/di
st-packages (from matplotlib>=3.1.3->sweetviz) (1.3.2)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dist-p
ackages (from matplotlib>=3.1.3->sweetviz) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/d
ist-packages (from matplotlib>=3.1.3->sweetviz) (4.59.0)
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.11/d
ist-packages (from matplotlib>=3.1.3->sweetviz) (1.4.8)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dis
t-packages (from matplotlib>=3.1.3->sweetviz) (25.0)
Requirement already satisfied: pillow>=6.2.0 in /usr/local/lib/python3.11/dist-
packages (from matplotlib>=3.1.3->sweetviz) (11.3.0)
Requirement already satisfied: pyparsing<3.1,>=2.3.1 in /usr/local/lib/python
3.11/dist-packages (from matplotlib>=3.1.3->sweetviz) (3.0.9)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.1
1/dist-packages (from matplotlib>=3.1.3->sweetviz) (2.9.0.post0)
Requirement already satisfied: mkl_fft in /usr/local/lib/python3.11/dist-packag
es (from numpy>=1.16.0->sweetviz) (1.3.8)
Requirement already satisfied: mkl_random in /usr/local/lib/python3.11/dist-pac
kages (from numpy>=1.16.0->sweetviz) (1.2.4)
Requirement already satisfied: mkl_umath in /usr/local/lib/python3.11/dist-pack
ages (from numpy>=1.16.0->sweetviz) (0.1.1)
Requirement already satisfied: mkl in /usr/local/lib/python3.11/dist-packages
(from numpy>=1.16.0->sweetviz) (2025.3.0)
Requirement already satisfied: tbb4py in /usr/local/lib/python3.11/dist-package
s (from numpy>=1.16.0->sweetviz) (2022.3.0)
Requirement already satisfied: mkl-service in /usr/local/lib/python3.11/dist-pa
ckages (from numpy>=1.16.0->sweetviz) (2.4.1)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-p
ackages (from pandas!=1.0.0,!=1.0.1,!=1.0.2,>=0.25.3->sweetviz) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dis
t-packages (from pandas!=1.0.0,!=1.0.1,!=1.0.2,>=0.25.3->sweetviz) (2025.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packa
ges (from python-dateutil>=2.7->matplotlib>=3.1.3->sweetviz) (1.17.0)
Requirement already satisfied: onemkl-license==2025.3.0 in /usr/local/lib/pytho
n3.11/dist-packages (from mkl->numpy>=1.16.0->sweetviz) (2025.3.0)
```

```
Requirement already satisfied: intel-openmp<2026,>=2024 in /usr/local/lib/pytho
n3.11/dist-packages (from mkl->numpy>=1.16.0->sweetviz) (2024.2.0)
Requirement already satisfied: tbb==2022.* in /usr/local/lib/python3.11/dist-pa
ckages (from mkl->numpy>=1.16.0->sweetviz) (2022.3.0)
Requirement already satisfied: tcmlib==1.* in /usr/local/lib/python3.11/dist-pa
ckages (from tbb==2022.*->mkl->numpy>=1.16.0->sweetviz) (1.4.0)
Requirement already satisfied: intel-cmplr-lib-rt in /usr/local/lib/python3.11/
dist-packages (from mkl_umath->numpy>=1.16.0->sweetviz) (2024.2.0)
Requirement already satisfied: intel-cmplr-lib-ur==2024.2.0 in /usr/local/lib/p
ython3.11/dist-packages (from intel-openmp<2026,>=2024->mkl->numpy>=1.16.0->swe
etviz) (2024.2.0)
Downloading sweetviz-2.3.1-py3-none-any.whl (15.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 15.1/15.1 MB 113.3 MB/s eta 0:00:00
00:01:01
Installing collected packages: sweetviz
Successfully installed sweetviz-2.3.1
```

In [30]:
```python
# Import SweetViz library
import sweetviz as sv

# Create and generate the report
sweet_report = sv.analyze(df)

# Save the report to an HTML file
sweet_report.show_html('sweetviz_report.html')
```

```
                                                |          | [  0%]   00:00 -> (?
left)
Report sweetviz_report.html was generated! NOTEBOOK/COLAB USERS: the web browse
r MAY not pop up, regardless, the report IS saved in your notebook/colab files.
```

Splitting data into train and test

In [31]:
```python
from sklearn.model_selection import train_test_split
```

In [32]:
```python
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=
```

In [33]:
```python
x_train.shape,x_test.shape,y_train.shape,y_test.shape
```

Out[33]:
```
((3971, 13), (993, 13), (3971,), (993,))
```

Model Building

SVR

In [34]:
```python
from sklearn.svm import SVR
```

```
svr_model = SVR(C=100, gamma='auto')
svr_model.fit(x_train, y_train)
```

Out[34]:

```
▾ SVR
SVR(C=100, gamma='auto')
```

In [35]:

```
svr_y_pred = svr_model.predict(x_test)
svr_y_pred
```

Out[35]:
```
array([10.83271764, 21.74237415, 15.86609117, 17.08694448, 34.67861414,
       12.89921523, 20.55586298, 14.14362001, 38.08076234, 29.34116863,
       11.4292084 , 24.36241052, 24.44931881, 18.81636781, 26.81437715,
       20.32388505, 34.67861414, 16.60523026, 32.15710186, 25.45728658,
       25.75210828, 20.5011394 , 32.15710186, 11.3644923 , 25.86037349,
       28.2706846 , 12.71658329, 11.93894251, 19.50142593, 13.58699287,
       28.77094576, 11.3644923 , 26.0476209 , 18.80236036, 31.89220764,
       19.25167428, 20.32388505, 22.96338281, 15.0998841 , 19.30013805,
       19.85724319, 20.03775134,  8.19982187, 13.74281038, 19.27949237,
       19.52714842, 22.44062852, 15.69957598, 47.34179262, 18.81636781,
       12.63971858, 18.07632365, 32.15710186, 28.77094576, 15.36615701,
       30.29409943, 12.71658329, 13.83942055, 13.91089584, 22.90420148,
       17.38411673, 20.43361573, 11.10612266, 33.77557823, 21.13397081,
       28.20287439, 26.0476209 , 31.87055658, 15.09469603, 19.85395157,
       13.74281038, 18.49422158, 18.04997377, 29.7700075 ,  9.43608331,
       13.82380573,  9.7933727 , 25.13634324, 11.4292084 ,  9.7933727 ,
       22.39222061, 16.50042913, 22.42197536, 17.05227528, 24.14735328,
       28.44974462, 31.70011335, 23.07078512, 29.58384414, 15.53577074,
       46.15156889, 34.3281688 , 22.44751428,  9.84642107, 25.86037349,
       22.25955536, 15.65466124, 24.90285407, 16.65571148, 19.27660722,
       19.50142593, 23.79765289, 32.19841693, 18.50018042, 22.29948183,
       20.03775134, 13.69954132, 26.81437715, 18.73130518, 25.42620858,
       14.36634303, 26.49999761, 23.17351379, 17.10061405, 42.49268603,
       34.94534024, 13.74281038, 18.20568811, 28.47758457, 11.4171773 ,
       18.81636781, 15.2808679 , 27.82101342,  7.18413594, 24.19921347,
       18.19252753, 17.51033243, 20.41217169, 14.82847122, 17.8993043 ,
       18.21441244, 23.07078512, 20.73735683, 21.0647405 , 22.92533978,
       13.90024689, 19.30027911, 15.18327562, 26.35650948, 13.58699287,
       11.93894251, 35.0605833 , 23.07078512, 17.56506601, 31.64983981,
       22.86589502, 33.10631751, 16.87845794, 18.04997377, 20.52519072,
       18.19252753, 44.20677325, 15.16255808, 13.91089584, 11.4292084 ,
       22.96338281, 17.04483938, 18.97671898, 41.562259  , 16.65571148,
       47.34179262,  6.97622499, 29.34116863, 24.09578199, 20.70065254,
       18.88556495, 42.49268603, 22.57946474, 20.89856362, 18.35057397,
       22.63670688, 21.74033456, 16.3162941 , 16.59955079, 13.27553473,
       26.74683343, 16.06220605, 14.60785918, 24.48307069, 34.3281688 ,
       16.60523026, 25.71068014, 28.53165876, 22.63670688, 25.13634324,
       35.0605833 , 43.76586053, 29.18915541, 27.1319371 , 18.26644217,
       19.78299567, 18.50018042, 13.44123194, 19.62758862, 23.03119065,
       25.18946894, 22.92533978, 16.26378812, 27.21576588, 21.26872297,
       26.49999761, 18.2048691 , 24.44931881, 17.65254379, 34.3281688 ,
       30.80064141, 41.39375686, 40.71214368, 21.04802916, 23.77834285,
       21.24878602, 18.97671898, 25.30877556, 34.88201641, 16.37342081,
       17.04483938, 13.82380573, 26.29270783, 15.85745451, 18.26644217,
       16.59955079, 40.71214368, 18.81301539, 14.36634303, 11.4171773 ,
       26.0476209 , 21.74033456, 13.72321034, 20.43361573, 31.41228921,
       19.94992127, 11.10612266, 20.5746747 , 22.82229966, 10.56302403,
       34.80016305, 26.7953202 , 19.62758862, 32.12043279, 31.1983926 ,
       20.41217169, 15.58470254, 26.55473637, 20.36531761, 20.01503482,
       35.60897906, 16.37342081, 17.05227528, 21.69833302, 42.76659574,
       20.6544542 , 19.49272266, 20.8977883 , 26.78151585, 34.7047176 ,
       20.69191356, 42.49268603, 19.80054496, 42.49268603, 16.65571148,
       28.6021434 , 35.0605833 , 24.90285407, 18.2048691 , 24.85560238,
       16.26378812, 20.8977883 ,  7.22014832, 22.12150251, 30.63275396,
```

35.29153465, 17.69004885, 16.3162941 , 30.14625094, 12.52076826,
31.69998447, 33.53698411, 22.69531162, 15.58470254, 22.54160725,
17.22636736, 27.24257845, 49.53283739, 21.20021402, 22.6415133 ,
20.24133986, 20.79631622, 18.88556495, 18.59563006, 18.80236036,
15.28100409, 22.57946474, 41.562259 , 31.69830434, 21.61753367,
13.72321034, 24.61519363, 26.93855376, 28.01879161, 25.60233898,
18.39113488, 23.29978785, 22.54160725, 18.25435116, 9.43608331,
21.61902417, 32.29974203, 42.76659574, 22.54160725, 19.50142593,
48.13579682, 20.5011394 , 21.69833302, 12.14348386, 31.70011335,
18.2048691 , 21.04802916, 19.80054496, 26.78151585, 18.25435116,
22.07957862, 15.09469603, 17.53146376, 31.69830434, 22.9388972 ,
22.47068639, 14.36634303, 34.48653781, 21.04802916, 20.79631622,
24.48307069, 9.43608331, 22.6415133 , 34.80016305, 21.69833302,
15.18327562, 12.39967819, 21.10715946, 12.33904594, 31.64983981,
24.66655964, 23.77834285, 17.73160386, 24.46250982, 23.69691255,
15.64789314, 30.31702644, 18.60202882, 25.30877556, 17.04483938,
22.28678649, 17.91303691, 13.7430132 , 10.83271764, 12.66720008,
15.09941942, 33.53698411, 22.74308974, 18.81301539, 23.17351379,
17.35035466, 12.71658329, 19.27949237, 50.10029423, 20.52519072,
22.25955536, 19.30027911, 24.48307069, 23.69691255, 36.02454191,
24.53256413, 41.64346181, 22.44062852, 18.87835996, 12.71658329,
26.61171838, 13.83942055, 10.07325151, 7.29996807, 19.51138125,
 8.19982187, 26.35650948, 17.69004885, 18.71602864, 23.17351379,
21.35227409, 22.03612183, 21.61902417, 21.10715946, 10.27771526,
23.95637409, 38.08076234, 15.64789314, 16.03389614, 23.95637409,
20.10996006, 9.47267737, 22.42197536, 11.3644923 , 24.53256413,
20.41217169, 24.99071763, 21.59940228, 9.7933727 , 21.74237415,
15.73142867, 24.45424801, 24.28856349, 22.28663999, 28.6021434 ,
31.1983926 , 16.06220605, 22.82229966, 12.52554379, 17.05227528,
17.21317561, 17.22636736, 30.31702644, 36.70166784, 21.13397081,
40.40964449, 32.29974203, 16.37342081, 28.47758457, 35.0605833 ,
18.15287306, 22.44001525, 20.79631622, 33.77557823, 27.83748935,
16.03389614, 23.0001886 , 30.80064141, 20.41217169, 26.0476209 ,
26.35650948, 9.84642107, 25.31627353, 19.25167428, 21.40033447,
23.48361643, 21.30632514, 13.99288862, 16.0431919 , 15.74848929,
 7.29996807, 18.85223471, 30.60169855, 34.7047176 , 14.82847122,
16.65961849, 32.19841693, 13.67730832, 12.63971858, 17.22636736,
24.90285407, 14.60785918, 48.13579682, 10.12871836, 20.50906892,
23.92466243, 21.10715946, 18.15619713, 16.26378812, 15.86609117,
20.98710216, 19.94604829, 17.10061405, 9.7551216 , 32.29974203,
18.35057397, 23.79765289, 22.07957862, 12.40798877, 24.66655964,
50.10029423, 12.92884089, 22.03612183, 31.64983981, 23.65728166,
23.60010471, 14.54670423, 21.60025828, 19.80054496, 16.80692555,
12.52554379, 40.17170111, 10.69222228, 7.80902057, 14.54670423,
19.08022913, 19.85395157, 19.30027911, 18.21441244, 39.81447799,
24.45424801, 12.71658329, 14.49322525, 26.93855376, 30.29409943,
22.90420148, 18.04997377, 47.34179262, 13.84539394, 12.89921523,
22.86589502, 9.7551216 , 48.13579682, 42.49268603, 14.14362001,
22.46040308, 41.71103371, 23.69691255, 25.10271237, 25.60233898,
23.46463361, 14.48285765, 32.9274784 , 19.62758862, 12.66720008,
23.48361643, 19.52714842, 16.4166901 , 19.08022913, 24.79161533,
18.04997377, 20.63876113, 19.86331663, 17.86498603, 18.50018042,
20.36531761, 17.91303691, 26.35650948, 23.29978785, 18.92530105,
12.52076826, 24.77558537, 26.56092718, 32.19841693, 26.55473637,

22.07957862, 52.50139808, 14.60785918, 19.78299567, 23.27841708,
22.9388972 , 18.01999712, 15.09941942, 9.7933727 , 15.58470254,
23.90337666, 30.14625094, 42.76659574, 31.69998447, 18.15287306,
25.20548894, 47.34179262, 4.90044579, 15.53577074, 17.0006952 ,
21.42346813, 32.9274784 , 24.79161533, 43.21067306, 21.30632514,
26.7154275 , 18.15287306, 26.35650948, 24.14735328, 29.97206905,
49.53283739, 15.23189622, 20.69191356, 18.81301539, 15.0998841 ,
20.35214576, 18.04997377, 19.78299567, 12.52554379, 17.38411673,
18.78403932, 31.87055658, 20.10996006, 18.2048691 , 15.499736 ,
25.4900774 , 24.04670543, 24.51716438, 21.13397081, 9.84642107,
13.58699287, 17.69004885, 35.29960872, 24.19921347, 39.4715385 ,
11.10612266, 25.20548894, 15.69957598, 29.0506548 , 17.35035466,
17.91303691, 19.52714842, 27.32749669, 21.79965849, 21.10715946,
24.66655964, 8.94706462, 14.20002561, 14.35590457, 36.70166784,
18.01999712, 43.39923843, 24.46812965, 34.48653781, 25.31627353,
32.12043279, 22.03612183, 20.98710216, 10.85090991, 19.30013805,
25.13634324, 17.91303691, 25.4900774 , 30.80064141, 14.53273827,
17.73160386, 20.6544542 , 13.7430132 , 20.6544542 , 22.44001525,
13.77743703, 30.63275396, 16.37943472, 13.39979373, 12.71658329,
24.09578199, 24.99071763, 13.07751697, 18.97671898, 10.27771526,
13.07751697, 22.40134613, 19.30027911, 22.46040308, 13.95790066,
22.54160725, 28.01811799, 28.77094576, 17.05227528, 26.93855376,
10.50677494, 40.51010864, 13.99288862, 19.50142593, 19.44947199,
12.33755921, 12.14348386, 22.06162792, 33.84734578, 15.36615701,
17.73160386, 28.2706846 , 14.53273827, 14.54670423, 18.87835996,
11.3644923 , 17.80767863, 33.79189223, 16.31520961, 22.69531162,
20.70065254, 24.36241052, 34.6930894 , 13.72321034, 21.59940228,
15.43782326, 34.99952063, 44.20677325, 21.11473535, 12.52076826,
19.85395157, 17.04483938, 13.91089584, 37.20266833, 24.09578199,
18.82023629, 18.35057397, 44.20677325, 26.87134277, 16.31520961,
20.52519072, 21.61753367, 25.86037349, 19.51138125, 28.20287439,
18.88556495, 24.99071763, 18.92530105, 15.39986857, 34.3281688 ,
22.12150251, 19.85395157, 26.35650948, 20.89985576, 30.29409943,
15.00837527, 13.7430132 , 17.05227528, 27.32749669, 13.67730832,
32.29974203, 15.57164942, 21.60025828, 27.83748935, 16.66125802,
 9.47267737, 18.17123653, 18.78403932, 20.32388505, 23.03119065,
17.65254379, 30.63275396, 22.90420148, 17.40004625, 19.86331663,
12.66720008, 15.39986857, 19.00041484, 19.94992127, 43.76586053,
19.52714842, 21.53626969, 23.27841708, 16.03389614, 22.44001525,
13.07273679, 21.13397081, 20.03775134, 20.55586298, 34.80016305,
11.4171773 , 15.73142867, 18.73130518, 23.17351379, 23.17351379,
14.49322525, 30.29409943, 23.69691255, 18.19252753, 35.60897906,
16.65961849, 15.09941942, 15.28804455, 28.27579001, 14.9051764 ,
22.31441807, 21.85826684, 22.49989738, 49.90026546, 17.80767863,
23.07078512, 25.18946894, 33.10631751, 10.69222228, 38.08076234,
15.86609117, 18.78354326, 20.79631622, 31.41228921, 16.87845794,
12.13341886, 14.52837954, 17.53146376, 23.04242779, 20.52519072,
14.82847122, 22.69531162, 17.80767863, 26.55473637, 48.13579682,
 4.90044579, 31.87055658, 28.38435856, 17.3354836 , 14.9051764 ,
13.91089584, 16.05245277, 23.90337666, 40.40964449, 35.60897906,
10.50677494, 21.24878602, 25.10271237, 20.70065254, 46.15156889,
16.3162941 , 31.41228921, 21.59940228, 20.10996006, 13.84539394,
22.29948183, 33.10631751, 31.69998447, 25.71068014, 18.19252753,
18.26644217, 20.86966499, 33.93744211, 29.77765422, 14.49322525,

```
       24.45424801, 26.87134277, 25.45728658, 33.77557823, 15.16255808,
       22.44062852, 15.73142867, 29.48970191, 24.85560238, 14.36333957,
       24.99071763, 47.16491352, 19.00041484, 12.33755921, 26.78151585,
       40.17170111, 34.7047176 , 20.79631622, 32.19841693, 13.82380573,
       14.54670423, 18.44775515, 18.88556495, 20.35214576, 18.85223471,
       13.39979373,  9.47267737, 19.40340141, 18.73130518,  7.22014832,
       13.83942055, 47.35404667, 19.94992127, 16.06220605, 15.74848929,
       16.4166901 , 18.82023629, 41.39375686,  9.43608331, 41.39375686,
       11.3644923 , 31.1983926 , 32.12043279, 13.07273679, 25.86037349,
       13.24645927, 14.35590457, 22.9388972 , 32.29974203, 17.60860547,
       18.92530105, 28.5297046 , 25.60233898, 15.09469603, 29.0506548 ,
        7.29996807, 41.71103371, 12.33904594, 25.42620858, 19.86331663,
       18.78542883, 26.78151585, 19.2688533 , 24.61519363, 22.47667334,
       24.90285407, 30.14625094, 23.04242779, 19.27660722, 13.99994089,
       21.74033456,  6.97622499, 18.50018042, 39.81447799, 34.80016305,
       18.2048691 , 23.79765289, 26.55473637, 24.36241052, 15.40916592,
       23.07078512, 12.52076826, 33.79189223, 23.29978785, 18.07632365,
       39.81447799, 17.22636736, 24.77558537, 23.44178895, 10.56302403,
       26.7953202 , 31.69830434, 33.53698411, 20.86966499, 12.80797545,
       39.4715385 , 10.50677494, 22.88490375, 24.46812965, 14.60785918,
       25.10271237,  7.80902057, 30.93183825, 30.31702644,  7.29996807,
       17.79562595, 40.17170111, 19.39997402, 12.71658329, 18.39113488,
       13.99288862, 24.19921347, 20.24133986, 19.30027911, 13.69954132,
       24.09578199, 25.60233898, 23.44178895, 24.85560238, 13.69954132,
       24.48307069, 34.88201641, 25.60233898, 26.78151585, 24.61519363,
       19.73837875, 21.42346813, 23.04242779, 15.85745451, 27.21576588,
       16.50042913, 32.19841693, 22.90420148, 13.79425564, 12.09646356,
       25.13634324, 23.46463361, 26.81437715, 43.21067306, 20.50906892,
       20.24133986, 32.29974203, 23.50021715, 40.40964449, 16.06220605,
       31.69830434, 19.62758862, 23.0001886 , 16.37943472, 16.3162941 ,
       12.89921523, 18.44775515, 26.35650948, 12.09646356, 21.07580978,
       32.12043279, 23.46463361, 13.50022103, 24.09578199, 13.72321034,
       15.69957598,  7.29996807, 14.53273827, 15.2808679 , 23.95637409,
       18.87835996, 28.77094576, 26.29270783, 21.20021402, 18.04997377,
       17.91303691, 10.56302403, 22.54160725, 20.24133986, 20.89985576,
       33.93744211, 29.0506548 , 26.61171838, 26.81437715, 23.07078512,
       24.09578199, 28.5297046 , 10.50677494])
```

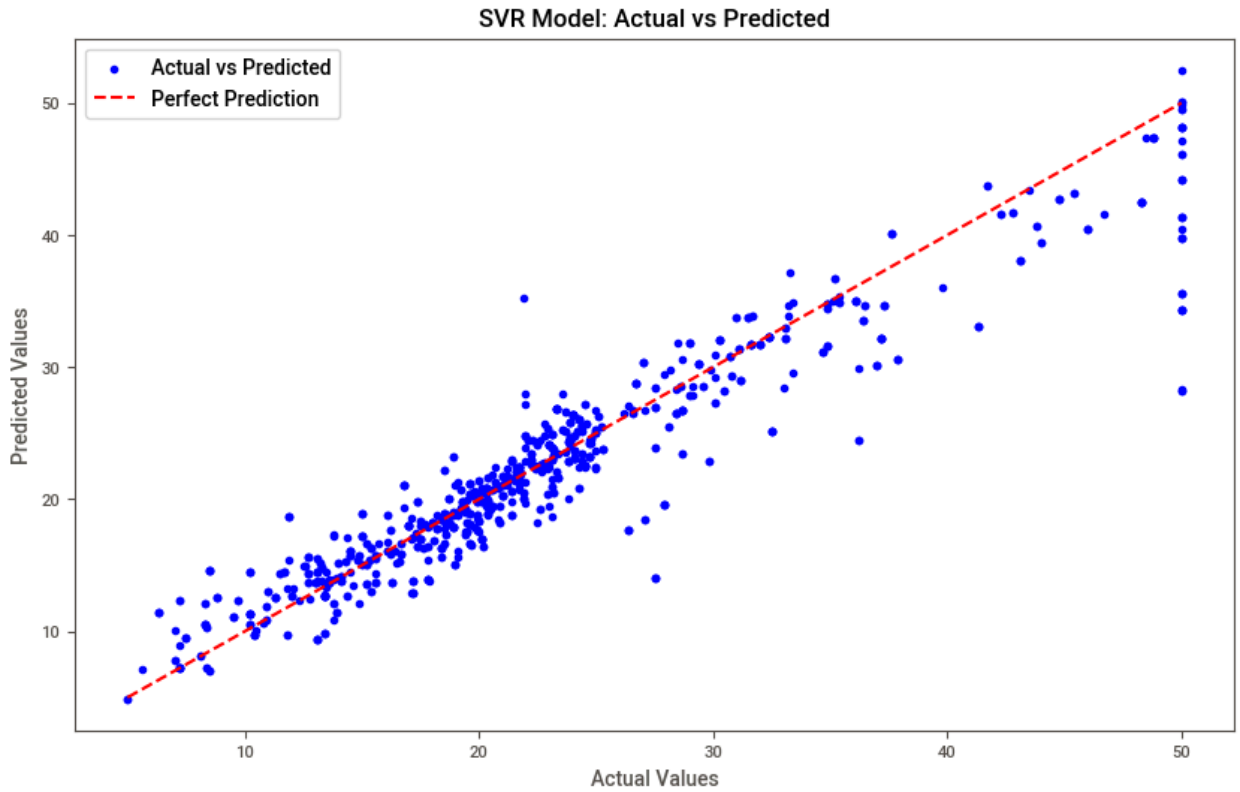
Evaluating the model

In [36]:

```python
# Evaluate the model's performance
from sklearn.metrics import mean_squared_error,r2_score,mean_absolute_error, m
mse_svr = mean_squared_error(y_test, svr_y_pred)
print('Mean squared error:', mse_svr)
r2_svr = r2_score(y_test, svr_y_pred)
print("R-squared (R2):", r2_svr)
mae_svr = mean_absolute_error(y_test, svr_y_pred)
print("Mean Absolute Error (MAE):", mae_svr)
rmse_svr= mean_squared_error(y_test, svr_y_pred, squared=False)
print("Root Mean Squared Error (RMSE):", rmse_svr)
```

```
Mean squared error: 10.206007549326214
R-squared (R2): 0.8861465551659926
Mean Absolute Error (MAE): 1.9659707232546655
Root Mean Squared Error (RMSE): 3.194684264418976
```

In [37]:
```
plt.figure(figsize=(10, 6))
plt.scatter(y_test, svr_y_pred, color='blue', label='Actual vs Predicted')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], linestyle='--
plt.title('SVR Model: Actual vs Predicted')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.legend()
plt.show()
```



We got 0.0039 as mse ,which indicates that the model is performing well.

Linear regression

In [38]:
```
from sklearn.linear_model import LinearRegression

lr=LinearRegression()

lr.fit(x_train,y_train)
```

Out[38]:

```
▾ LinearRegression
LinearRegression()
```

In [39]:

```
lr_y_pred=lr.predict(x_test)
```

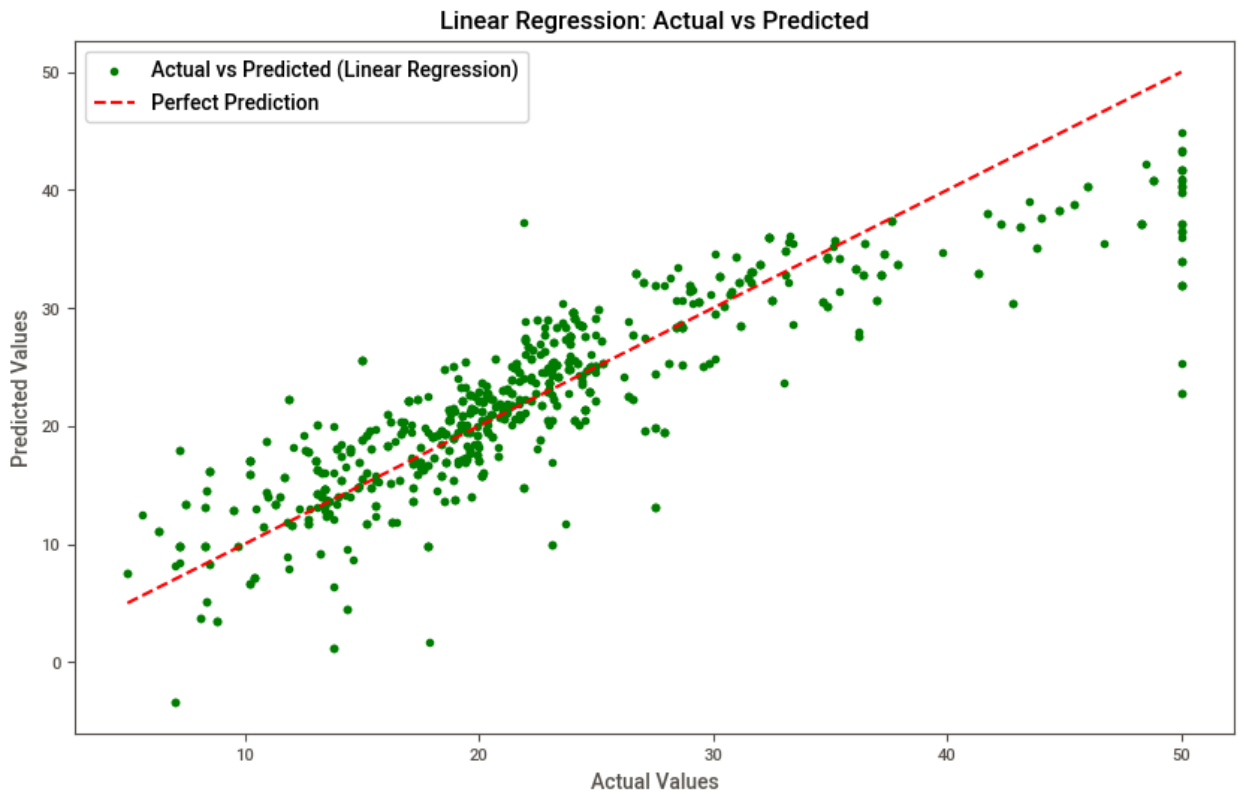
In [40]:

```
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

mse_linear = mean_squared_error(y_test, lr_y_pred)
print('Mean squared error:', mse_linear)  # <-- Use mse_linear here
r2_linear = r2_score(y_test, lr_y_pred)
print("R-squared (R2):", r2_linear)
mae_linear = mean_absolute_error(y_test, lr_y_pred)
print("Mean Absolute Error (MAE):", mae_linear)
rmse_linear = mean_squared_error(y_test, lr_y_pred, squared=False)
print("Root Mean Squared Error (RMSE):", rmse_linear)
```

```
Mean squared error: 21.969719811298113
R-squared (R2): 0.7549160854070344
Mean Absolute Error (MAE): 3.3382245238695503
Root Mean Squared Error (RMSE): 4.6871867694063685
```

In [41]:

```
plt.figure(figsize=(10, 6))
plt.scatter(y_test, lr_y_pred, color='green', label='Actual vs Predicted (Line
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], linestyle='--
plt.title('Linear Regression: Actual vs Predicted')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.legend()
plt.show()
```



Descision Tree Regression

In [42]:
```
from sklearn.tree import DecisionTreeRegressor

dt=DecisionTreeRegressor()

dt.fit(x_train,y_train)
```

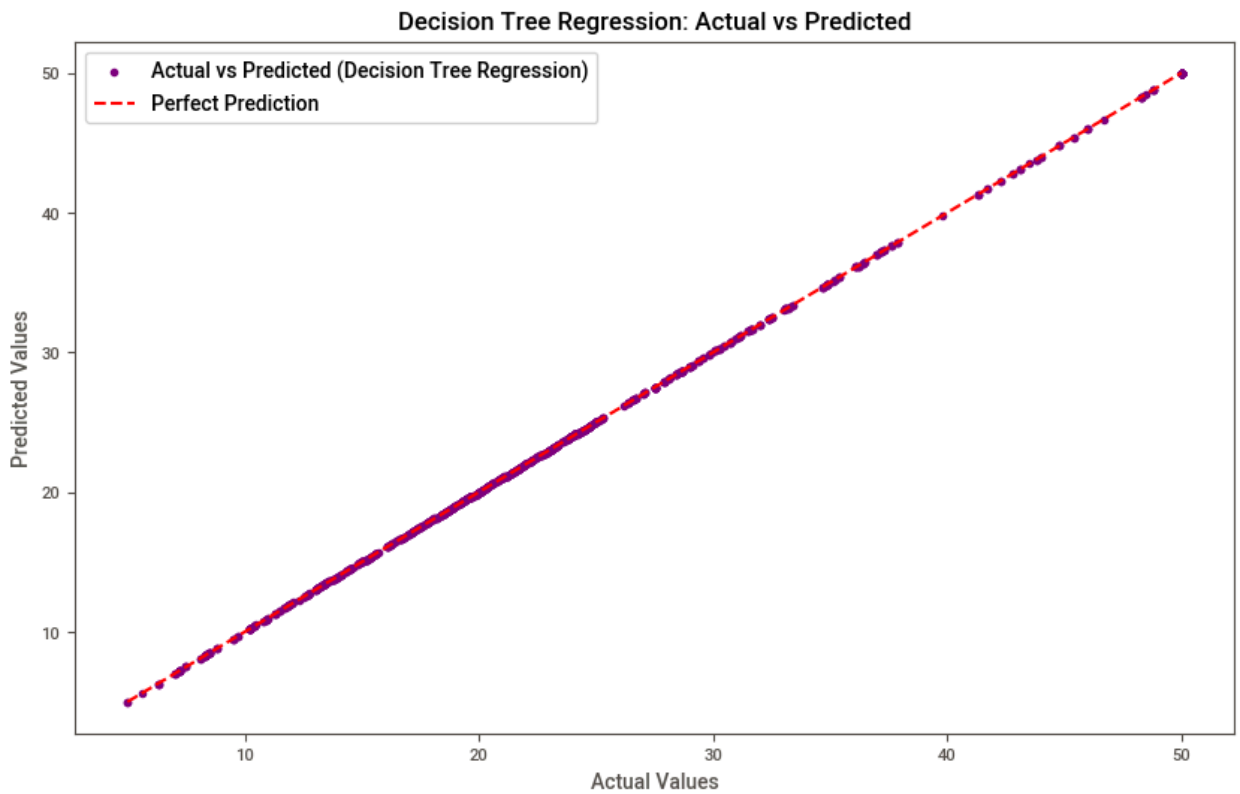
Out[42]:
```
▾ DecisionTreeRegressor
DecisionTreeRegressor()
```

In [43]:
```
dt_y_pred=dt.predict(x_test)
```

In [44]:
```
mse_decision_tree = mean_squared_error(y_test, dt_y_pred)
print('Mean squared error:', mse_decision_tree)
r2_decision_tree = r2_score(y_test, dt_y_pred)
print("R-squared (R2):", r2_decision_tree)
mae_decision_tree = mean_absolute_error(y_test, dt_y_pred)
print("Mean Absolute Error (MAE):", mae_decision_tree)
rmse_decision_tree= mean_squared_error(y_test, dt_y_pred, squared=False)
print("Root Mean Squared Error (RMSE):", rmse_decision_tree)
```

```
Mean squared error: 7.71065855596691e-30
R-squared (R2): 1.0
Mean Absolute Error (MAE): 1.7584680494767838e-15
Root Mean Squared Error (RMSE): 2.7768072594198736e-15
```

In [45]:
```
plt.figure(figsize=(10, 6))
plt.scatter(y_test, dt_y_pred, color='purple', label='Actual vs Predicted (Dec
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], linestyle='--
plt.title('Decision Tree Regression: Actual vs Predicted')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.legend()
plt.show()
```



KNN Regression

In [47]:
```
# Using K-Nearest Neighbors regression
from sklearn.neighbors import KNeighborsRegressor

knn_model = KNeighborsRegressor(n_neighbors=5)

knn_model.fit(x_train, y_train)

knn_y_pred = knn_model.predict(x_test)
```

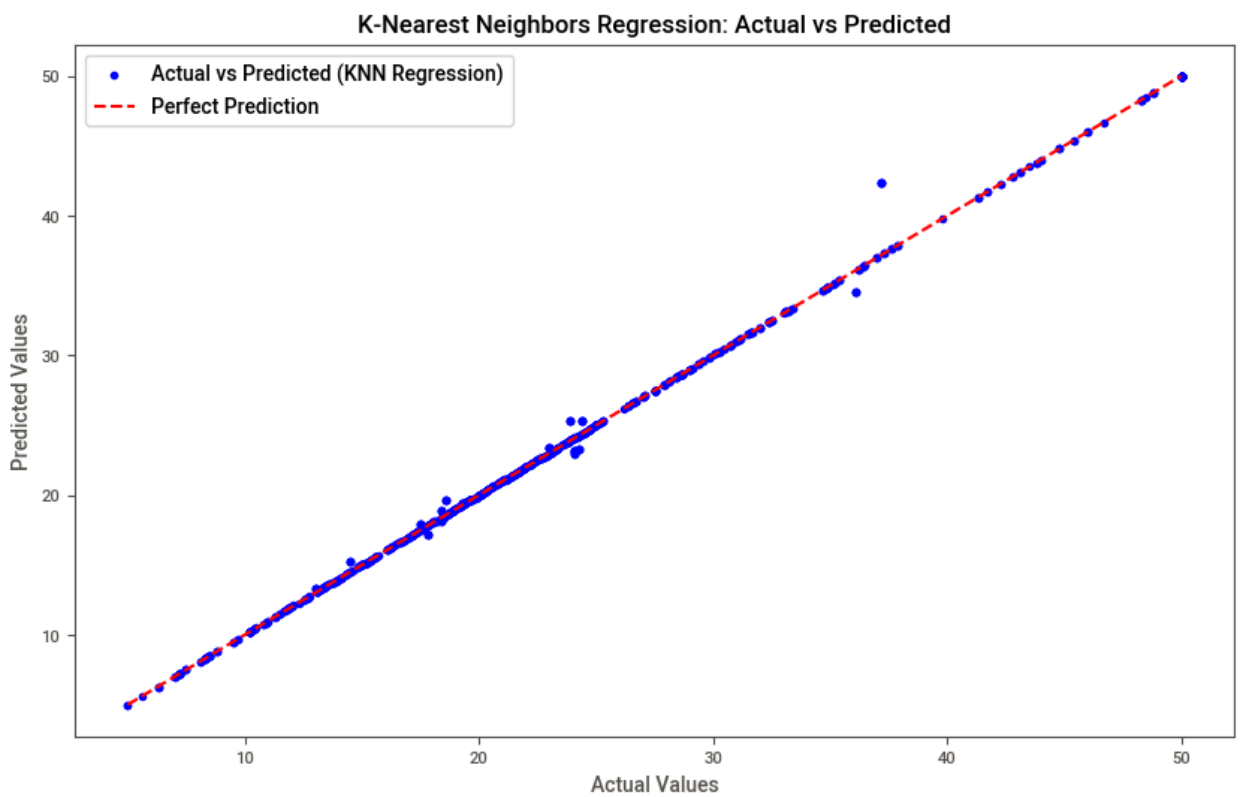
In [48]:
```
# Evaluate the KNN model's performance
mse_knn = mean_squared_error(y_test, knn_y_pred)
print('Mean squared error:', mse_knn)
r2_knn = r2_score(y_test, knn_y_pred)
print("R-squared (R2):", r2_knn)
mae_knn = mean_absolute_error(y_test, knn_y_pred)
print("Mean Absolute Error (MAE):", mae_knn)
rmse_knn = mean_squared_error(y_test, knn_y_pred, squared=False)
```

```
print("Root Mean Squared Error (RMSE):", rmse_knn)
```

```
Mean squared error: 0.18118267875125899
R-squared (R2): 0.9979788108111437
Mean Absolute Error (MAE): 0.07609264853977843
Root Mean Squared Error (RMSE): 0.4256555870081573
```

In [49]:

```
plt.figure(figsize=(10, 6))
plt.scatter(y_test, knn_y_pred, color='blue', label='Actual vs Predicted (KNN
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], linestyle='--
plt.title('K-Nearest Neighbors Regression: Actual vs Predicted')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.legend()
plt.show()
```



Bayesian Linear Regression

In [50]:

```
from sklearn.linear_model import BayesianRidge

# Creating and fitting the Bayesian Ridge Regression model
bayesian_regressor = BayesianRidge()

bayesian_regressor.fit(x_train,y_train)
```

Out[50]:

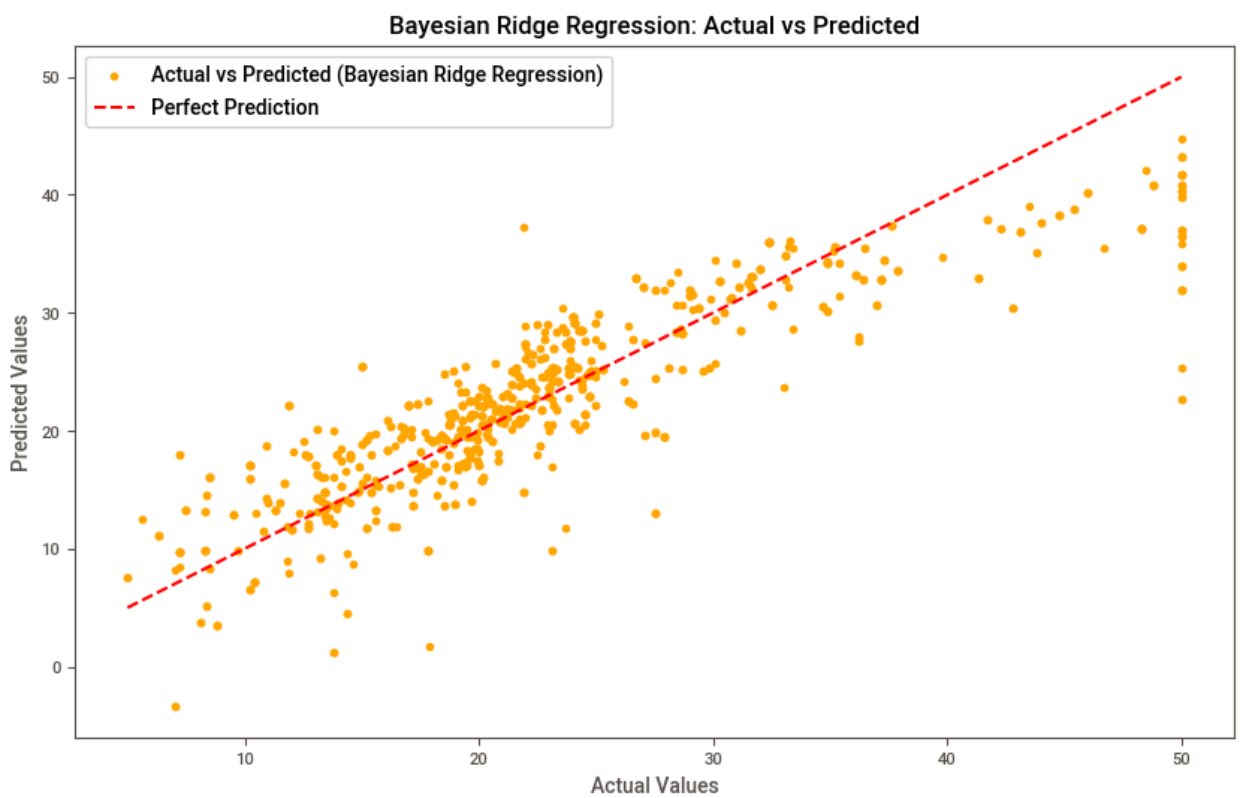
```
▾ BayesianRidge
BayesianRidge()
```

In [51]:
```
# Make predictions on the test set
bayesian_y_pred = bayesian_regressor.predict(x_test)
```

In [52]:
```
# Compute the evaluation metrics
mse_bayesian= mean_squared_error(y_test, bayesian_y_pred)
print('Mean squared error:', mse_bayesian)
r2_bayesian = r2_score(y_test, bayesian_y_pred)
print("R-squared (R2):", r2_bayesian)
mae_bayesian = mean_absolute_error(y_test, bayesian_y_pred)
print("Mean Absolute Error (MAE):", mae_bayesian)
rmse_bayesian = mean_squared_error(y_test, bayesian_y_pred, squared=False)
print("Root Mean Squared Error (RMSE):",rmse_bayesian)
```

```
Mean squared error: 21.977844119444296
R-squared (R2): 0.7548254544267157
Mean Absolute Error (MAE): 3.3376907367252873
Root Mean Squared Error (RMSE): 4.688053340080965
```

In [53]:
```
plt.figure(figsize=(10, 6))
plt.scatter(y_test, bayesian_y_pred, color='orange', label='Actual vs Predicte
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], linestyle='--
plt.title('Bayesian Ridge Regression: Actual vs Predicted')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.legend()
plt.show()
```



Elastic Net Regression

In [54]:
```
from sklearn.linear_model import ElasticNet

alpha = 0.5  # Regularization parameter
l1_ratio = 0.5  # Mixing parameter, 0 for L2 penalty, 1 for L1 penalty
elastic_net = ElasticNet(alpha=alpha, l1_ratio=l1_ratio)
elastic_net.fit(x_train, y_train)
```

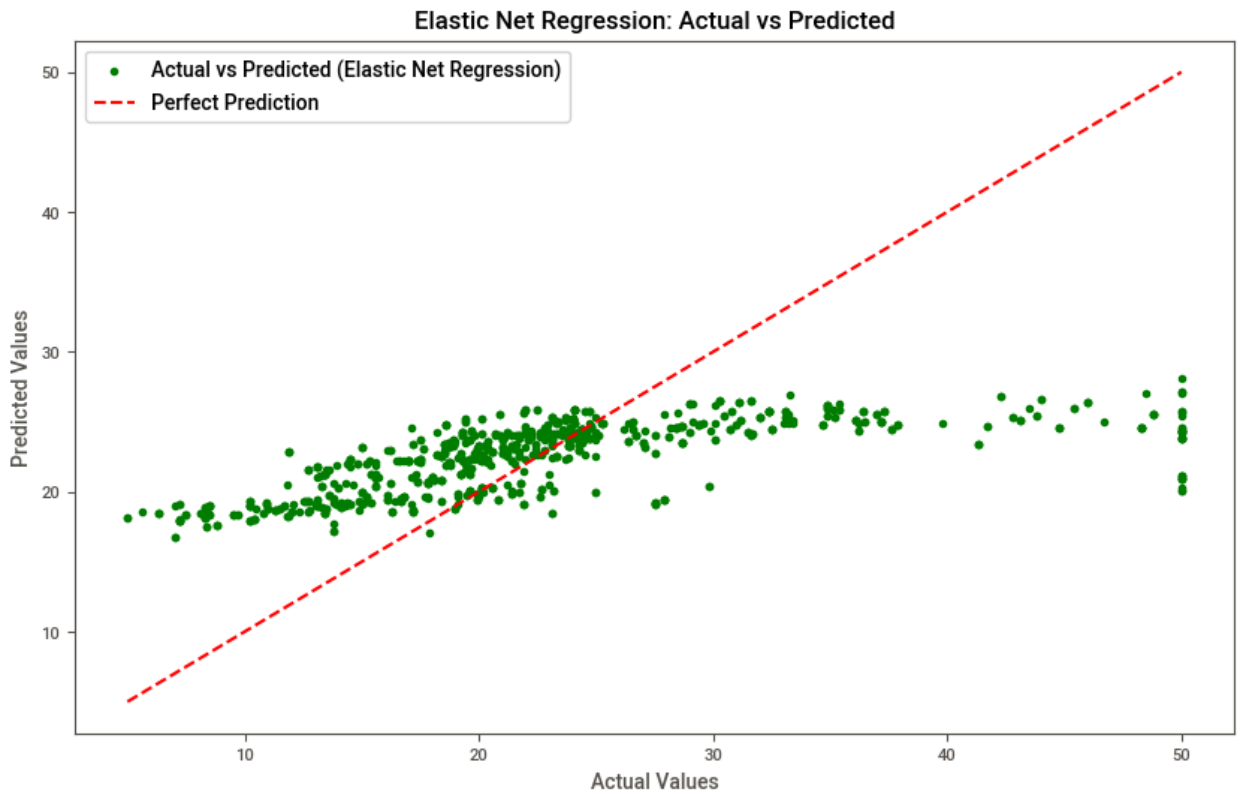
Out[54]:
```
▾ ElasticNet
ElasticNet(alpha=0.5)
```

In [55]:
```
y_pred_Elastic = elastic_net.predict(x_test)
```

In [56]:
```
# Compute the evaluation metrics
mse_elastic_net = mean_squared_error(y_test, y_pred_Elastic)
print('Mean squared error:', mse_elastic_net)
r2_elastic_net= r2_score(y_test, y_pred_Elastic)
print("R-squared (R2):", r2_elastic_net)
mae_elastic_net = mean_absolute_error(y_test, y_pred_Elastic)
print("Mean Absolute Error (MAE):", mae_elastic_net)
rmse_elastic_net = mean_squared_error(y_test, y_pred_Elastic, squared=False)
print("Root Mean Squared Error (RMSE):",rmse_elastic_net)
```

```
Mean squared error: 64.68855061122338
R-squared (R2): 0.2783647971245091
Mean Absolute Error (MAE): 5.630318213614379
Root Mean Squared Error (RMSE): 8.042919284142007
```

In [57]:
```
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred_Elastic, color='green', label='Actual vs Predicted
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], linestyle='--
plt.title('Elastic Net Regression: Actual vs Predicted')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.legend()
plt.show()
```



Negative R2 value suggests that this model is not appropriate to use for this dataset

Lasso L1 Regression

In [58]:
```
from sklearn import linear_model

lasso_reg = linear_model.Lasso(alpha=50, max_iter=100, tol=0.1)

lasso_reg.fit(x_train,y_train)
```

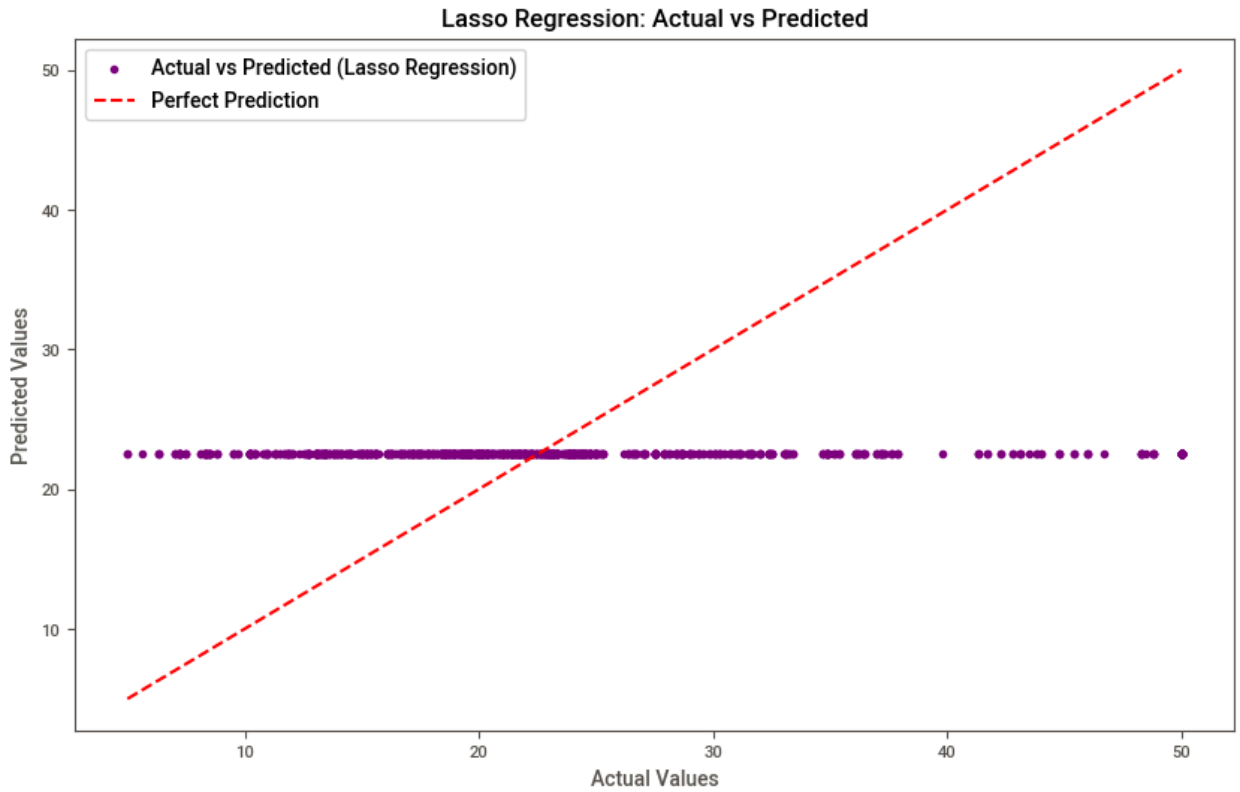
Out[58]:
```
▾ Lasso
Lasso(alpha=50, max_iter=100, tol=0.1)
```

In [59]:
```
y_pred_l1 = lasso_reg.predict(x_test)
```

In [60]:
```
# Compute the evaluation metrics
mse_lasso = mean_squared_error(y_test, y_pred_l1)
print('Mean squared error:', mse_lasso)
r2_lasso = r2_score(y_test, y_pred_l1)
print("R-squared (R2):", r2_lasso)
mae_lasso = mean_absolute_error(y_test, y_pred_l1)
print("Mean Absolute Error (MAE):", mae_lasso)
rmse_lasso = mean_squared_error(y_test, y_pred_l1, squared=False)
print("Root Mean Squared Error (RMSE):",rmse_lasso)
```

```
Mean squared error: 89.66976508983525
R-squared (R2): -0.0003139429000280636
Mean Absolute Error (MAE): 6.8832180843847
Root Mean Squared Error (RMSE): 9.469412077306345
```

In [61]:
```
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred_l1, color='purple', label='Actual vs Predicted (Las
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], linestyle='--
plt.title('Lasso Regression: Actual vs Predicted')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.legend()
plt.show()
```



considering the R2 value we assume this model is not suitable for this data

L2 Ridge Regression

In [62]:
```
from sklearn.linear_model import Ridge

ridge_reg= Ridge(alpha=50, max_iter=100, tol=0.1)

ridge_reg.fit(x_train,y_train)
```

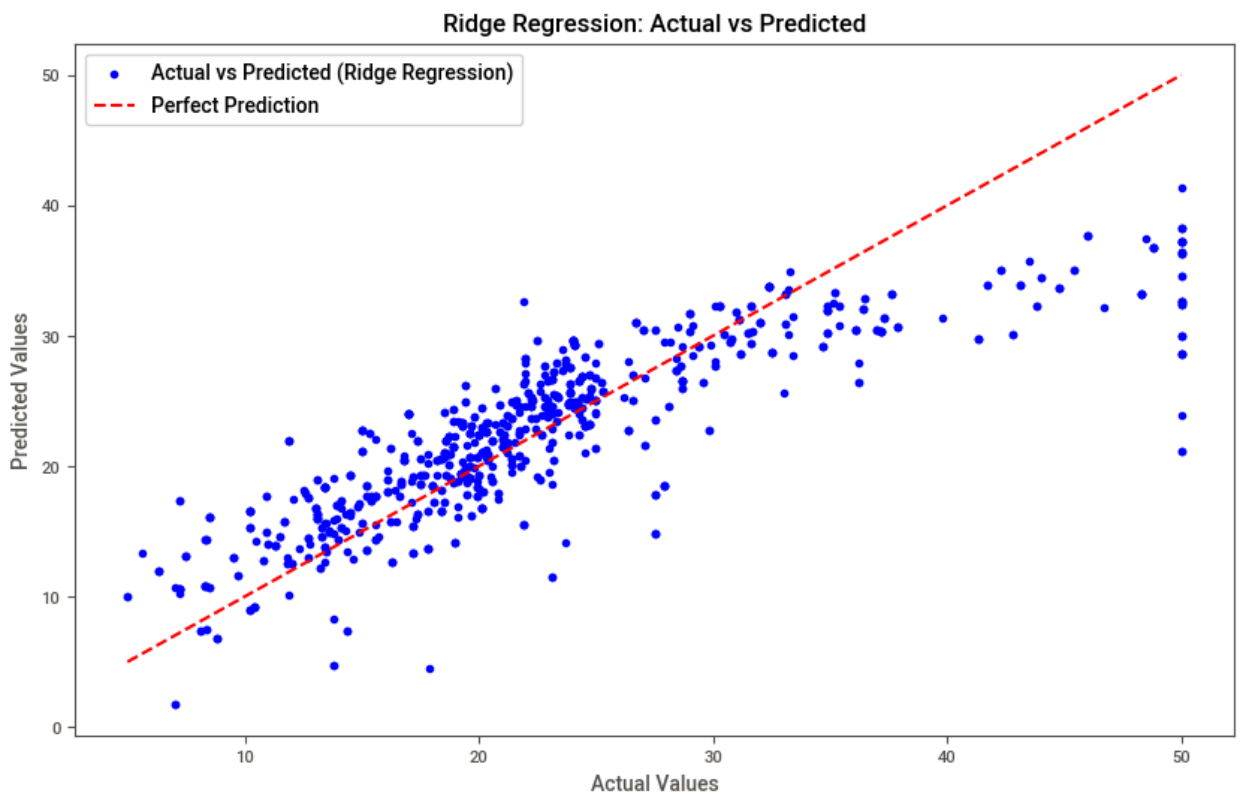
Out[62]:
```
▾ Ridge
Ridge(alpha=50, max_iter=100, tol=0.1)
```

In [63]:
```
y_pred_l2 = ridge_reg.predict(x_test)
```

In [64]:
```
# Compute the evaluation metrics
mse_ridge= mean_squared_error(y_test, y_pred_l2)
print('Mean squared error:', mse_ridge)
r2_ridge = r2_score(y_test, y_pred_l2)
print("R-squared (R2):", r2_ridge)
mae_ridge = mean_absolute_error(y_test, y_pred_l2)
print("Mean Absolute Error (MAE):", mae_ridge)
rmse_ridge = mean_squared_error(y_test, y_pred_l2, squared=False)
print("Root Mean Squared Error (RMSE):",rmse_ridge)
```

```
Mean squared error: 27.01032220300086
R-squared (R2): 0.6986854836207579
Mean Absolute Error (MAE): 3.601141708734159
Root Mean Squared Error (RMSE): 5.197145582240395
```

In [65]:
```
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred_l2, color='blue', label='Actual vs Predicted (Ridge
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], linestyle='--
plt.title('Ridge Regression: Actual vs Predicted')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.legend()
plt.show()
```



GBM Gradient boosting machine

In [66]:
```
from sklearn.ensemble import GradientBoostingRegressor

gb_regressor = GradientBoostingRegressor(n_estimators=100, learning_rate=0.1,

gb_regressor.fit(x_train,y_train)
```

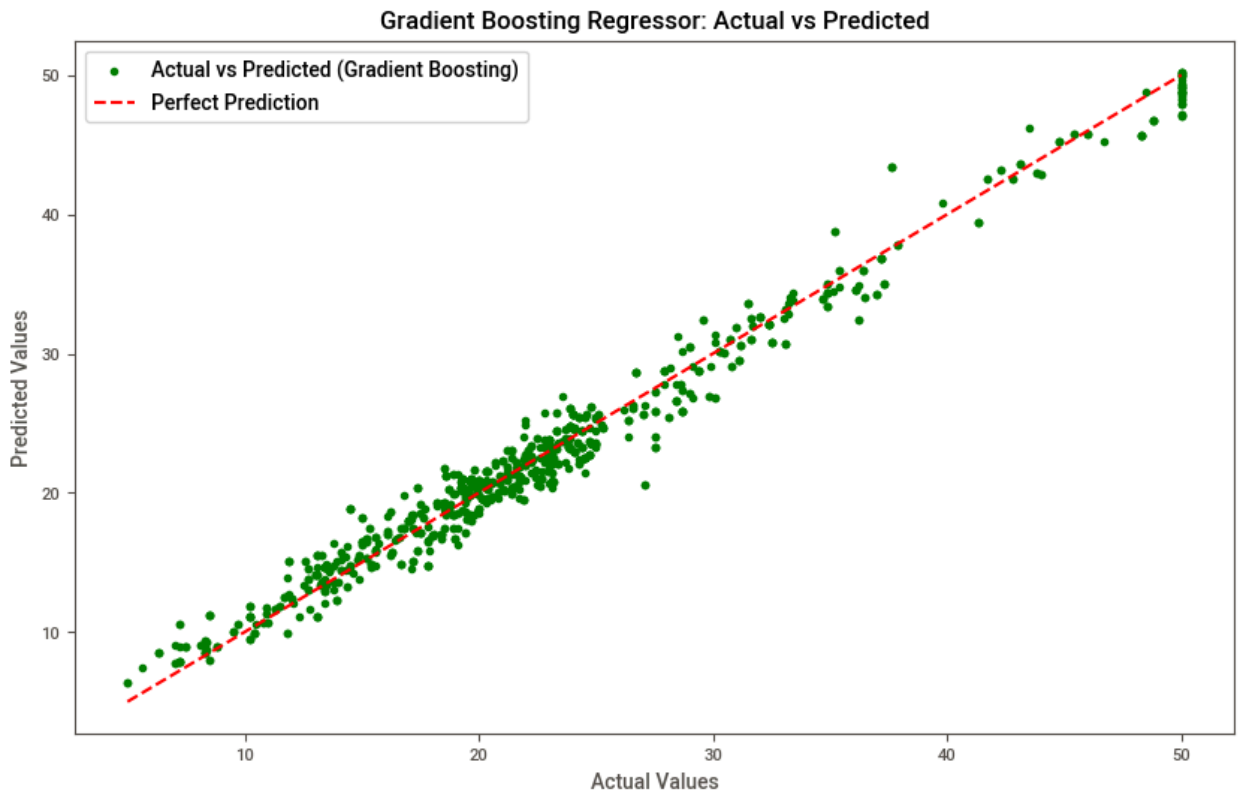
Out[66]:
```
▾ GradientBoostingRegressor
GradientBoostingRegressor(random_state=42)
```

In [67]:
```
y_pred_gbm = gb_regressor.predict(x_test)
```

In [68]:
```
# Compute the evaluation metrics
mse_gradient_boosting = mean_squared_error(y_test, y_pred_gbm)
print('Mean squared error:', mse_gradient_boosting)
r2_gradient_boosting = r2_score(y_test, y_pred_gbm)
print("R-squared (R2):", r2_gradient_boosting)
mae_gradient_boosting = mean_absolute_error(y_test, y_pred_gbm)
print("Mean Absolute Error (MAE):", mae_gradient_boosting)
rmse_gradient_boosting = mean_squared_error(y_test, y_pred_gbm, squared=False)
print("Root Mean Squared Error (RMSE):",rmse_gradient_boosting)
```

```
Mean squared error: 2.128908557974412
R-squared (R2): 0.9762508922425804
Mean Absolute Error (MAE): 1.1379875534057422
Root Mean Squared Error (RMSE): 1.4590779821429738
```

In [69]:
```
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred_gbm, color='green', label='Actual vs Predicted (Gra
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], linestyle='--
plt.title('Gradient Boosting Regressor: Actual vs Predicted')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.legend()
plt.show()
```



Random forest Regression

In [70]:
```
from sklearn.ensemble import RandomForestRegressor

random_forest_regressor = RandomForestRegressor(n_estimators=100, random_state

random_forest_regressor.fit(x_train,y_train)
```

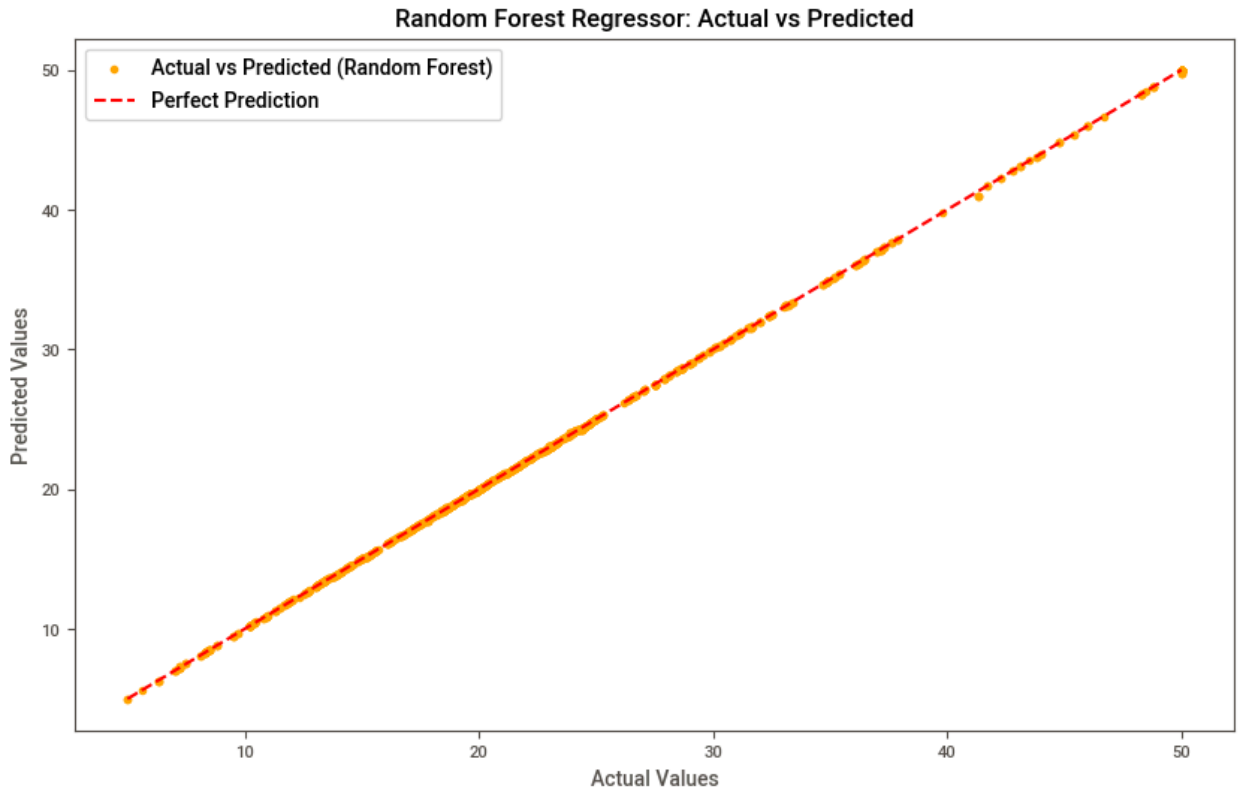
Out[70]:
```
▾ RandomForestRegressor
RandomForestRegressor(random_state=42)
```

In [71]:
```
y_pred_randomforest = random_forest_regressor.predict(x_test)
```

In [72]:
```
# Compute the evaluation metrics
mse_random_forest = mean_squared_error(y_test, y_pred_randomforest)
print('Mean squared error:', mse_random_forest )
r2_random_forest = r2_score(y_test, y_pred_randomforest)
print("R-squared (R2):", r2_random_forest )
mae_random_forest = mean_absolute_error(y_test, y_pred_randomforest)
print("Mean Absolute Error (MAE):", mae_random_forest )
rmse_random_forest = mean_squared_error(y_test, y_pred_randomforest, squared=F
print("Root Mean Squared Error (RMSE):",rmse_random_forest )
```

```
Mean squared error: 0.0010764612286000327
R-squared (R2): 0.9999879915022094
Mean Absolute Error (MAE): 0.008175226586121425
Root Mean Squared Error (RMSE): 0.03280946858149386
```

In [73]:
```
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred_randomforest, color='orange', label='Actual vs Prec
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], linestyle='--
plt.title('Random Forest Regressor: Actual vs Predicted')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.legend()
plt.show()
```



Ada Boost Regression

In [74]:
```
from sklearn.ensemble import AdaBoostRegressor

ada_boost_reg = AdaBoostRegressor(n_estimators=50, random_state=42)

ada_boost_reg.fit(x_train,y_train)
```

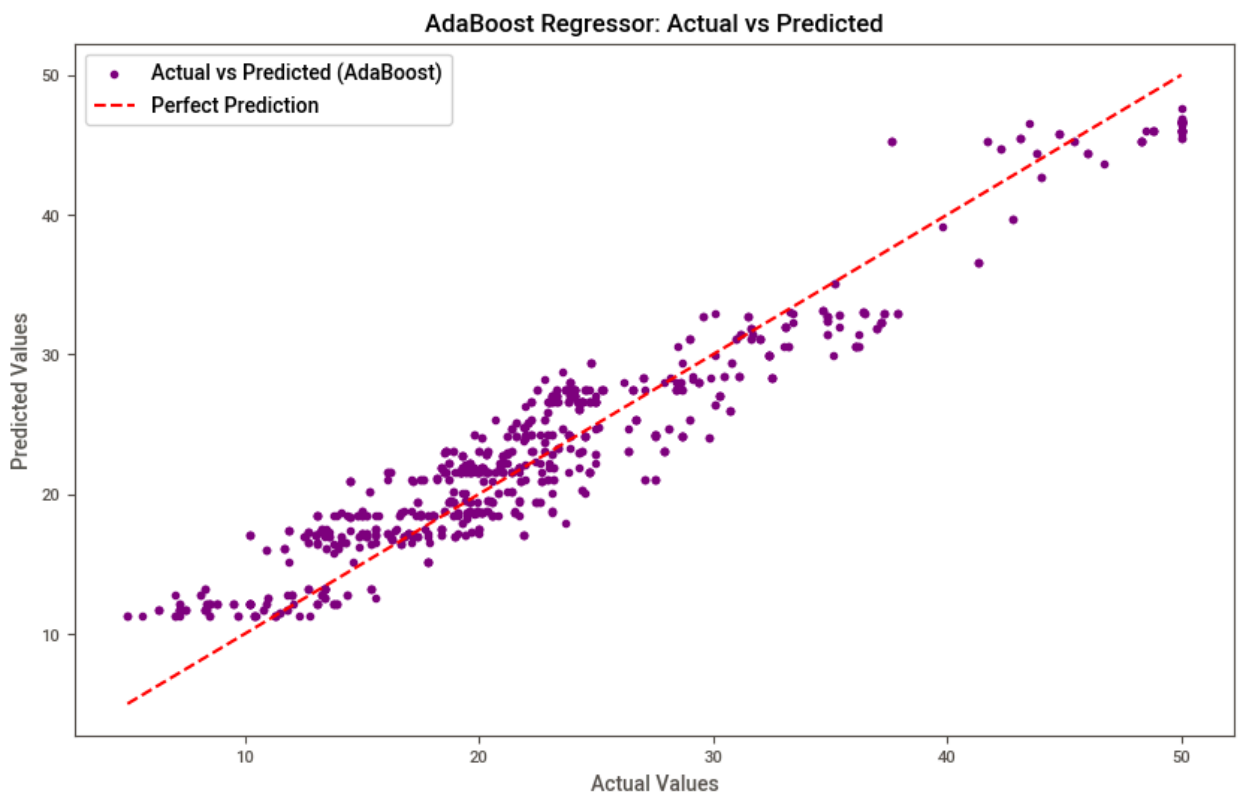
Out[74]:
```
▾ AdaBoostRegressor
AdaBoostRegressor(random_state=42)
```

In [75]:
```
y_pred_adaboost = ada_boost_reg.predict(x_test)
```

In [76]:
```
# Compute the evaluation metrics
mse_adaboost = mean_squared_error(y_test, y_pred_adaboost)
print('Mean squared error:', mse_adaboost)
r2_adaboost= r2_score(y_test, y_pred_adaboost)
print("R-squared (R2):", r2_adaboost)
mae_adaboost = mean_absolute_error(y_test, y_pred_adaboost)
print("Mean Absolute Error (MAE):", mae_adaboost)
rmse_adaboost = mean_squared_error(y_test, y_pred_adaboost, squared=False)
print("Root Mean Squared Error (RMSE):",rmse_adaboost)
```

```
Mean squared error: 8.236521316530135
R-squared (R2): 0.9081172220573559
Mean Absolute Error (MAE): 2.4145661392393065
Root Mean Squared Error (RMSE): 2.8699340265117828
```

In [77]:
```
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred_adaboost, color='purple', label='Actual vs Predicte
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], linestyle='--
plt.title('AdaBoost Regressor: Actual vs Predicted')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.legend()
plt.show()
```



Poisson regression

In [78]:
```
from sklearn.linear_model import PoissonRegressor
```

```
poisson_reg = PoissonRegressor()

poisson_reg.fit(x_train,y_train)
```

Out[78]:

```
▾ PoissonRegressor
PoissonRegressor()
```

In [79]:

```
y_pred_poisson = poisson_reg.predict(x_test)
```

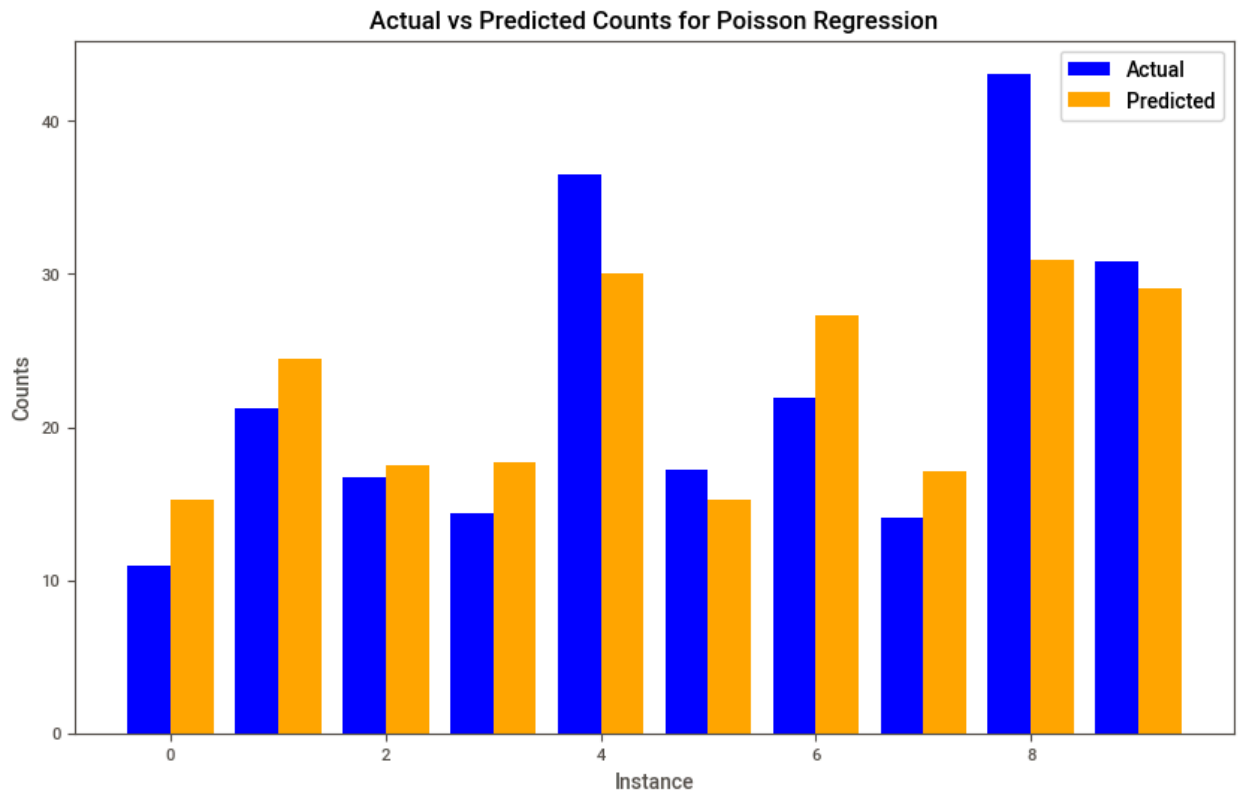
In [80]:

```
# Compute the evaluation metrics
mse_poisson = mean_squared_error(y_test, y_pred_poisson)
print('Mean squared error:', mse_poisson)
r2_poisson  = r2_score(y_test, y_pred_poisson)
print("R-squared (R2):", r2_poisson)
mae_poisson = mean_absolute_error(y_test, y_pred_poisson)
print("Mean Absolute Error (MAE):", mae_poisson)
rmse_poisson  = mean_squared_error(y_test, y_pred_poisson, squared=False)
print("Root Mean Squared Error (RMSE):",rmse_poisson)
```

```
Mean squared error: 35.85459906331409
R-squared (R2): 0.6000228692002112
Mean Absolute Error (MAE): 4.066812971133038
Root Mean Squared Error (RMSE): 5.987870995881098
```

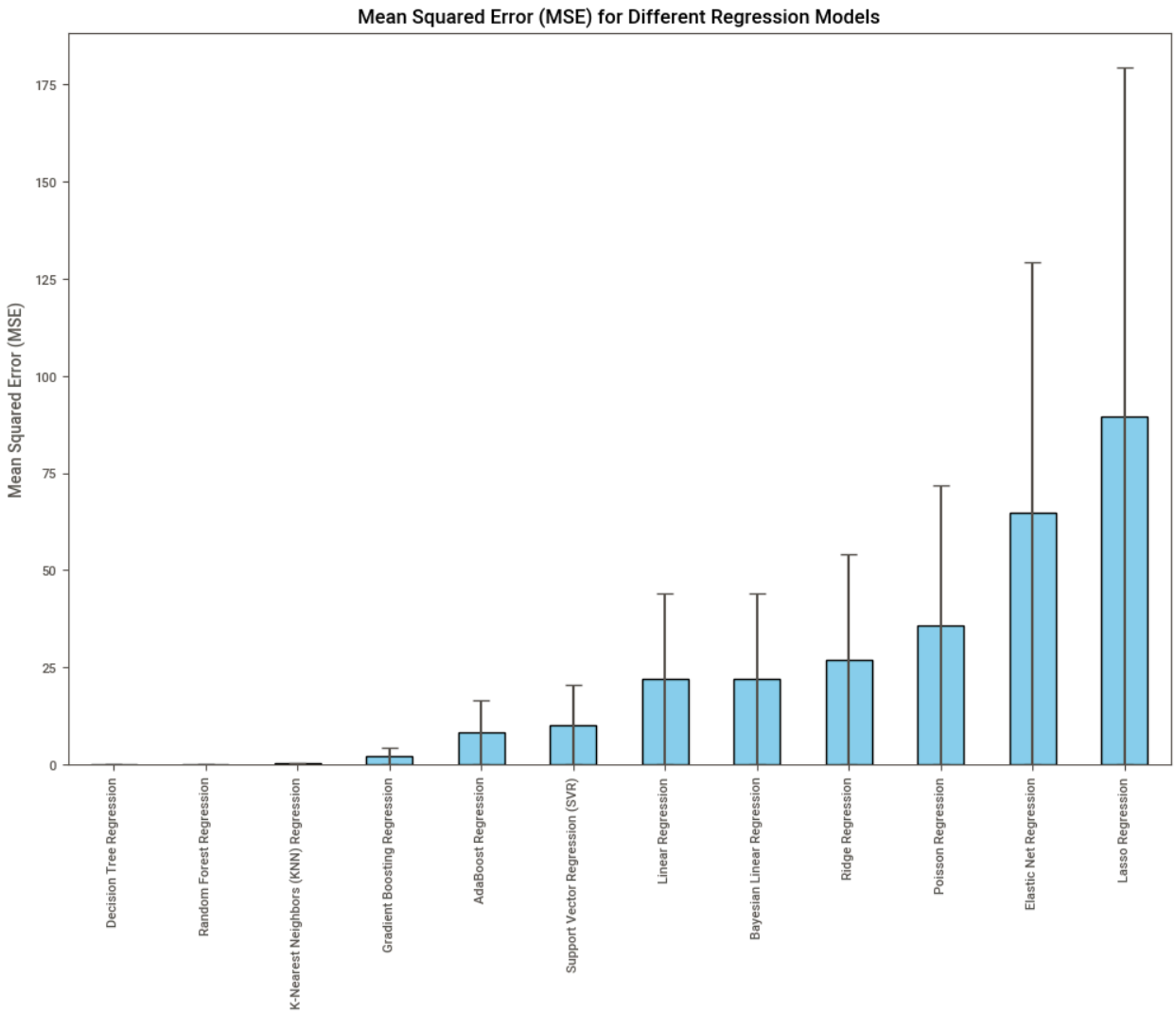
In [81]:

```
indices = np.arange(len(y_test[:10]))

# Plot actual and predicted counts
plt.figure(figsize=(10, 6))
plt.bar(indices - 0.2, y_test[:10], width=0.4, label='Actual', color='blue')
plt.bar(indices + 0.2, y_pred_poisson[:10], width=0.4, label='Predicted', colo
plt.xlabel('Instance')
plt.ylabel('Counts')
plt.title('Actual vs Predicted Counts for Poisson Regression')
plt.legend()
plt.show()
```



In [82]:
```
models = {
    'Linear Regression': {'mse': mse_linear, 'r2': r2_linear, 'mae': mae_linea
    'Ridge Regression': {'mse': mse_ridge, 'r2': r2_ridge, 'mae': mae_ridge, '
    'Lasso Regression': {'mse': mse_lasso, 'r2': r2_lasso, 'mae': mae_lasso, '
    'Elastic Net Regression': {'mse': mse_elastic_net, 'r2': r2_elastic_net, '
    'Support Vector Regression (SVR)': {'mse': mse_svr, 'r2': r2_svr, 'mae': m
    'Decision Tree Regression': {'mse': mse_decision_tree, 'r2': r2_decision_t
    'Random Forest Regression': {'mse': mse_random_forest, 'r2': r2_random_for
    'Gradient Boosting Regression': {'mse': mse_gradient_boosting, 'r2': r2_gr
    'AdaBoost Regression': {'mse': mse_adaboost, 'r2': r2_adaboost, 'mae': mae
    'K-Nearest Neighbors (KNN) Regression': {'mse': mse_knn, 'r2': r2_knn, 'ma
    'Bayesian Linear Regression': {'mse': mse_bayesian, 'r2': r2_bayesian, 'ma
    'Poisson Regression': {'mse': mse_poisson, 'r2': r2_poisson, 'mae': mae_po
}
```

In [83]:
```
# Convert the 'models' dictionary to a DataFrame for easier plotting
df = pd.DataFrame.from_dict(models, orient='index')

fig, ax = plt.subplots(figsize=(12, 8))
df['mse'].sort_values().plot(kind='bar', color='skyblue', edgecolor='black', a
ax.set_title('Mean Squared Error (MSE) for Different Regression Models')
ax.set_ylabel('Mean Squared Error (MSE)')

plt.show()
```

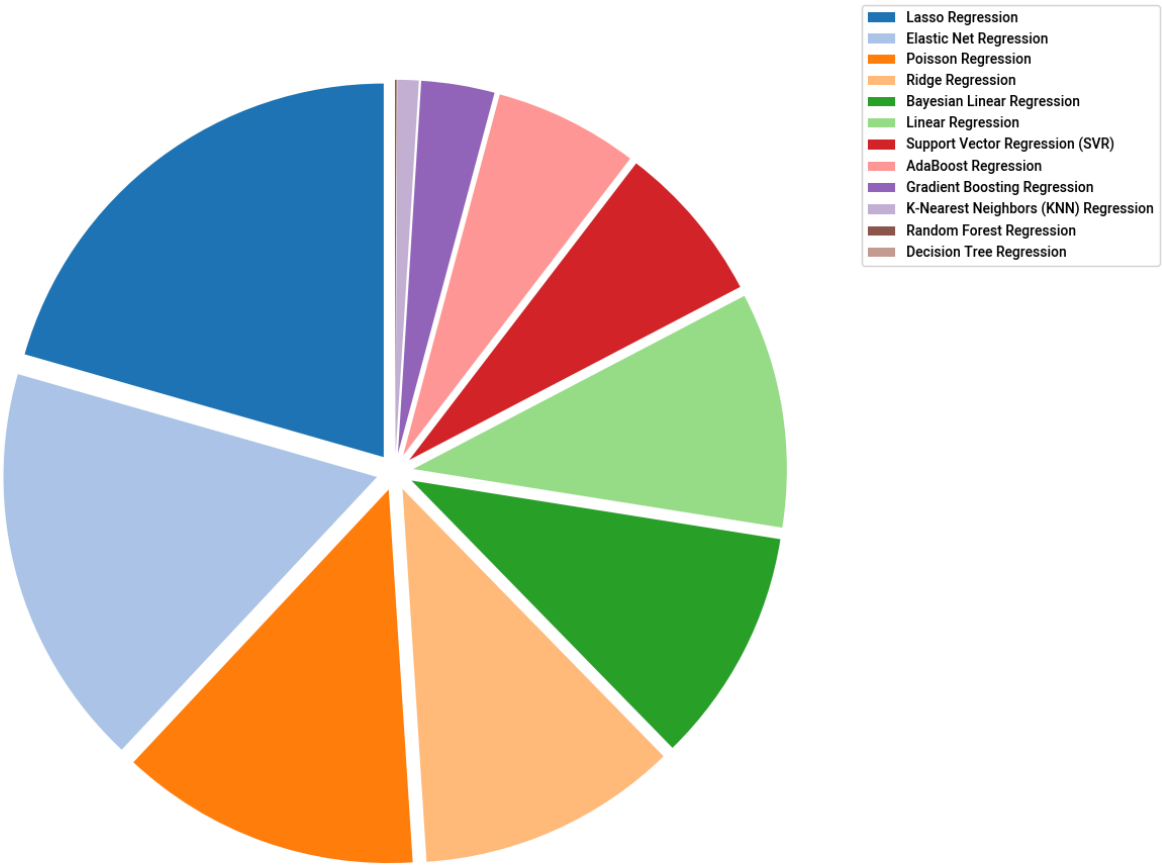


In [84]:

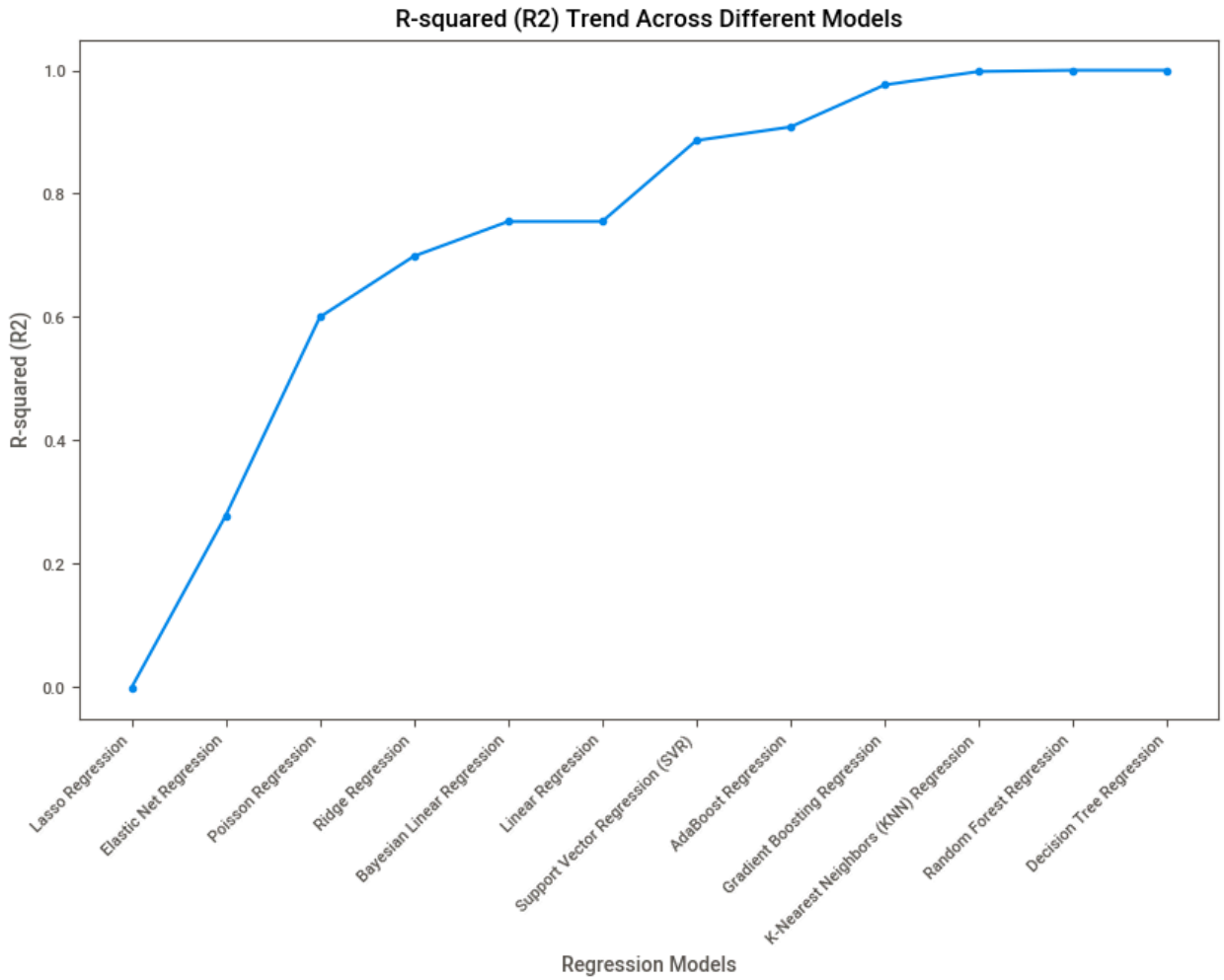
```
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.colors as colors

# Convert the dictionary to a DataFrame
df = pd.DataFrame.from_dict(models, orient='index')

# Define a list of colors for each model using a different colormap ('tab20')
colors = plt.cm.tab20.colors

# Sort the DataFrame by RMSE values in descending order
df = df.sort_values(by='rmse', ascending=False)

# Define the explode values for each model
explode = [0.05] * len(df)

# Create a pie chart with custom colors and explode
fig, ax = plt.subplots(figsize=(12, 12))
wedge, labels, autopct = ax.pie(
    df['rmse'],
    labels=None, # Suppress labels inside the pie chart
```

```
    autopct='', # Suppress percentage labels
    startangle=90,
    colors=colors[:len(df)], # Use only the needed number of colors
    explode=explode, # Explode values
)

# Create a custom legend outside of the pie chart
ax.legend(wedge, df.index, loc='upper left', bbox_to_anchor=(1, 1), borderaxes

# Set the title and remove the y-axis label
ax
```

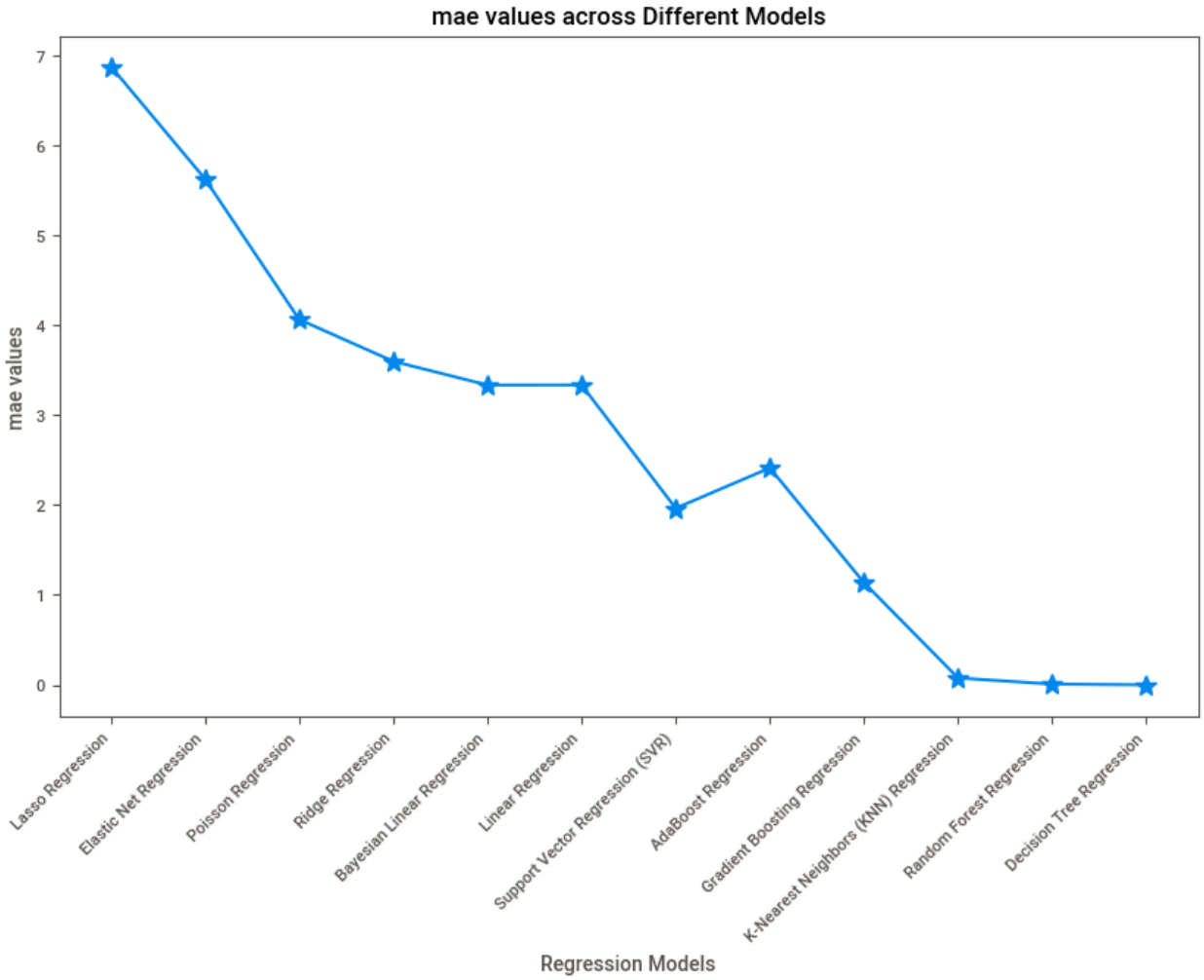
Out[84]: <Axes: >



In [85]:
```
plt.figure(figsize=(10, 6))
plt.plot(df.index, df['r2'], marker='o')
plt.title('R-squared (R2) Trend Across Different Models')
plt.xlabel('Regression Models')
plt.ylabel('R-squared (R2)')
plt.xticks(rotation=45, ha='right')
plt.show()
```



In [86]:
```
plt.figure(figsize=(10, 6))
plt.plot(df.index, df['mae'], marker='*',markersize=10)
plt.title('mae values across Different Models')
plt.xlabel('Regression Models')
plt.ylabel('mae values')
plt.xticks(rotation=45, ha='right')
plt.show()
```



Based on the above graphs,we can say random forest is performing well. so we will select random forest for our project.

Predict New Values

In [87]:
```
df
```

Out[87]:

	mse	r2	mae	rmse
Lasso Regression	8.966977e+01	-0.000314	6.883218e+00	9.469412e+00
Elastic Net Regression	6.468855e+01	0.278365	5.630318e+00	8.042919e+00
Poisson Regression	3.585460e+01	0.600023	4.066813e+00	5.987871e+00
Ridge Regression	2.701032e+01	0.698685	3.601142e+00	5.197146e+00
Bayesian Linear Regression	2.197784e+01	0.754825	3.337691e+00	4.688053e+00
Linear Regression	2.196972e+01	0.754916	3.338225e+00	4.687187e+00
Support Vector Regression (SVR)	1.020601e+01	0.886147	1.965971e+00	3.194684e+00
AdaBoost Regression	8.236521e+00	0.908117	2.414566e+00	2.869934e+00
Gradient Boosting Regression	2.128909e+00	0.976251	1.137988e+00	1.459078e+00
K-Nearest Neighbors (KNN) Regression	1.811827e-01	0.997979	7.609265e-02	4.256556e-01
Random Forest Regression	1.076461e-03	0.999988	8.175227e-03	3.280947e-02
Decision Tree Regression	7.710659e-30	1.000000	1.758468e-15	2.776807e-15

In [88]:

```
crim_new=float(input("Enter Crime rate in zone :"))
zn_new=float(input("Enter Proportion of residential land in the zone :"))
indus_new=float(input("Enter Proportion of non-retail business acres in zone :
chas_new=int(input("Bound by Charles River, Enter (Yes-1/No-0):"))
nox_new=float(input("Enter Nitric oxides concentration (parts per 10 million)
rm_new=float(input("Enter Average number of rooms per dwelling :"))
age_new=float(input("Enter Proportion of owner-occupied units built prior to 1
dis_new=float(input("Enter Weighted distances to five Boston employment center
rad_new=int(input("Enter Index of accessibility to radial highways :"))
tax_new=float(input("Enter Full-value property tax rate per $10,000 in the zor
ptratio_new= float(input("Enter Pupil-teacher ratio in the zone :"))
b_new= float(input("Enter proportion of Black residents in the zone :"))
lstat_new= float(input("Enter Percentage of lower status of the population :")

prdeiction=random_forest_regressor.predict(minmax.transform([[crim_new,zn_new,
print(f"Prediction Value : {prdeiction}")
```

```
Prediction Value : [8.344]
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/base.py:439: UserWarning: X doe
s not have valid feature names, but MinMaxScaler was fitted with feature names
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/base.py:439: UserWarning: X doe
s not have valid feature names, but RandomForestRegressor was fitted with featu
re names
  warnings.warn(
```

In [ ]: